

《港口管理》课程设计

港口计划期装卸作业技术方案编制与通过能力核算

专业班级：物流工程 2023 级 3 班

学生姓名：栾智健

学号：2220232872

任课教师：李宝德

完成日期：2026 年 07 月 03 日

大连海事大学

Dalian Maritime University

摘 要

本文以某港计划期 500 万吨货运任务为对象，围绕多货种、多流向、多装卸工艺和多资源约束下的港口装卸作业组织问题，建立货流网络模型与工艺选择混合整数规划模型，完成装卸作业技术方案编制、主要生产指标计算、通过能力核算和薄弱环节识别。研究首先对任务书表一进行数据校核，发现逐项合计为 419.0 万吨，与题干 500 万吨控制量不一致，因此采用 500 万吨校正口径作为主计算口径，并保留表一原始 419 万吨口径进行对照。随后将船、车、库、场抽象为网络节点，将车—场、场—船、船—车、船—库、库—车等操作过程抽象为有向边，在边上记录货量、工艺效率、配工人数、机械台时、作业工时和库场占用。在初始方案基础上，本文进一步以操作边工艺选择为决策变量，采用 Gurobi 求解工艺选择 MILP，比较规则枚举方案、情景方案和工艺优化方案。计算结果表明，初始方案总操作量为 917.9 万吨，操作系数为 1.836，直接换装比重为 16.42%；资源利用率排序显示，跨运车等机械以及部分库场环节负荷较高。优化分析表明，提高直接换装比例和缩短堆存期能够降低库场压力，工艺切换能够缓解部分机械瓶颈，但在现有资源条件下仍需要增配关键机械或进一步调整生产组织。

关键词：港口管理；货流网络模型；混合整数规划；装卸工艺；通过能力；薄弱环节

目 录

摘 要.....	I
第 1 章 绪论.....	1
1.1 研究背景.....	1
1.2 课程设计问题的工程含义.....	1
1.3 研究对象与数据基础.....	2
1.4 技术路线与方法框架.....	2
第 2 章 已知条件与数据预处理.....	2
2.1 货运量资料.....	2
2.2 装卸工艺资料.....	3
2.3 机械与人力资源条件.....	3
2.4 货运量双口径校核.....	3
2.5 计算参数与假设.....	4
第 3 章 港口货流网络建模.....	4
3.1 货流网络节点与边.....	4
3.2 货流路径拆分规则.....	4
3.3 操作过程展开方法.....	5
3.4 工艺方案匹配规则.....	5
3.5 货流网络模型图与说明.....	6
第 4 章 工艺选择优化模型与算法.....	6
4.1 问题抽象.....	6
4.2 决策变量.....	7
4.3 目标函数.....	7
4.4 约束条件.....	8
4.5 求解算法与求解器.....	8
4.6 算法流程与结果校验.....	9
第 5 章 装卸作业技术方案编制.....	11
5.1 初始方案.....	11

5.2 MILP 优化方案.....	12
5.3 分货类工艺方案.....	12
5.4 技术操作标准.....	14
5.5 方案比较.....	15
第 6 章 主要指标计算.....	15
6.1 吞吐量、操作量与操作系数.....	15
6.2 直接换装与机械化作业比重.....	15
6.3 同时开头量.....	15
6.4 机械台时、机种需要量与利用率.....	16
6.5 工时效率与劳动生产率.....	17
6.6 平均舱时量、车时量和车辆数.....	17
第 7 章 通过能力核算与薄弱环节分析.....	19
7.1 泊位通过能力.....	19
7.2 仓库与堆场通过能力.....	19
7.3 机械与人力通过能力.....	19
7.4 集疏运能力.....	20
7.5 薄弱环节识别.....	21
第 8 章 情景分析与资源改进建议.....	22
8.1 情景设置.....	22
8.2 情景结果比较.....	22
8.3 改进措施建议.....	23
结 论.....	25
参考文献.....	26
附录 A 主要计算程序.....	29
附录 B 货流拆分明细表.....	43
附录 C 机械台时统计表.....	43
附录 D 通过能力核算表.....	44

第 1 章 绪论

1.1 研究背景

港口装卸系统是港口生产组织的核心环节。货物从船舶、车辆进入港内后，可能直接换装，也可能经过仓库、前方堆场或后方堆场中转。不同货类的包装形式、件重、成组方式和装卸机械不同，会使同一吨货物在港内形成不同次数的操作量和不同强度的资源占用。对于计划期装卸作业技术方案而言，单纯列出货类和运量并不能说明资源是否匹配，必须进一步分析货物经过哪些作业节点、形成哪些操作过程，以及这些过程消耗多少机械台时和作业工时。

本课程设计给定的计划期货运量为 500 万吨，货类包括生铁、矿石、钢材、水泥、大米、海盐、集装箱杂货等，流向覆盖国内进出口和国外进出口。该任务具有典型的港口生产管理特征：一方面，直接换装比例会影响库场压力和操作系数；另一方面，机械数量、司机人数、班制、库场面积和堆存期又共同限制港口通过能力。因此，本设计不宜停留在装卸工艺表的机械填列，而应把货流、工艺和资源放在同一模型中计算。

1.2 课程设计问题的工程含义

从工程管理角度看，货流拆分是基础，决定货物是否直接换装以及是否进入库场；工艺选择是核心，决定每条操作边的效率、配工和机械台时；主要指标计算是结果，反映装卸组织的总体水平；通过能力核算是校验，用于判断泊位、库场、机械、人力等资源是否足以支撑计划任务；薄弱环节分析则是管理结论，指向后续调度调整和资源配置优化。上述环节具有递进关系，前一环节的口径若不清楚，后一环节的指标就难以解释。

因此，本文将课程设计理解为一个具有工程约束的数学建模问题：在给定货运量、路径比例、工艺参数和资源能力条件下，先构建可追溯的货流网络，再按操作边计算资源需求，最后通过统一的资源利用率指标识别瓶颈，并设置情景和优化模型进行比较。

1.3 研究对象与数据基础

研究对象为某港计划期装卸作业系统。基础资料包括任务书表一货运量资料、表二货类—操作过程—配机—配工—效率资料、表三机类—现有台数—司机人数—理论利用率资料，以及泊位数、仓库面积、前方堆场面积、后方堆场面积和装卸工人数等全局参数。由于任务书未给出船期、车辆到达分布和不同泊位专业化分工，本文在计算假设中明确采用标准车辆载重、平均堆存期和班制折算方法。

1.4 技术路线与方法框架

本文技术路线分为七步：第一步进行资料整理和双口径货运量校核；第二步按直接换装比例、库场路径和特殊备注拆分货流；第三步将路径展开为操作边并建立货流网络；第四步按表二匹配工艺方案，计算作业小时、作业工时和机械台时；第五步计算主要生产指标；第六步核算泊位、库场、机械、人力和集疏运通过能力；第七步进行情景分析和 MILP 工艺选择优化。

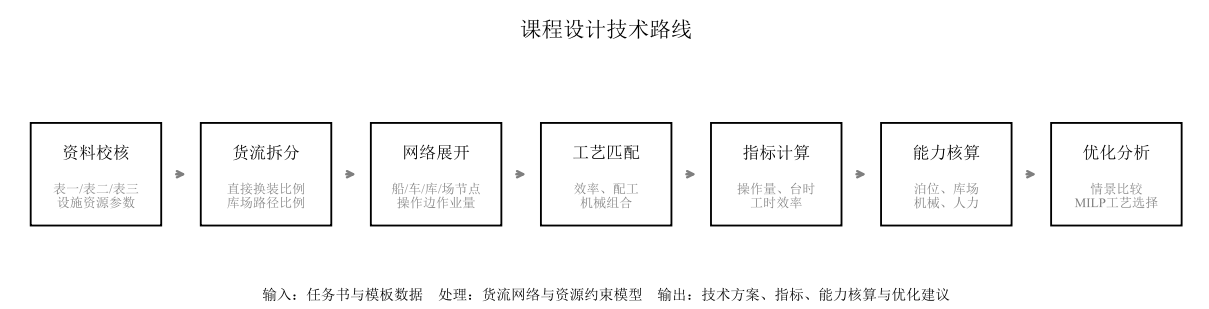


图 1-1 课程设计技术路线图

第 2 章 已知条件与数据预处理

2.1 货运量资料

任务书表一给出了各流向、各货类的原始运量、包装件重、工艺路径和备注。为便于后续计算，本文将其结构化为 cargo_flows 表，字段包括流向、货类、工艺货类、原始运量、校正后运量、包装件重、任务书路径和备注。

表 2-1 货运量资料及比例校正表

流向	货类	工艺货类	原始值(万吨)	500 万吨口径(万吨)	任务书路径	备注
国内出口	杂货	集装箱	35.00	41.77	车—场—船	集装箱杂货

国内进口	生铁	生铁	90.00	107.40	车—场—船	电磁吸盘卸 60%，卸车后全部成组
	水泥	水泥	15.00	17.90	车—库—船	卸车后全部货板
	纯碱	纯碱	19.00	22.67	车—场—船	成组
	棉花	棉花	2.40	2.86	船—车	直接换装
	矿石	矿石	36.00	42.96	船—车；船—场—车	50%直接换装
国外出口	杂货	杂货	10.40	12.41	船—车；船—库—车	25%直接换装（不成组），其余货盘成组
	海盐	海盐	9.60	11.46	车—船；车—场—船	50%直接换装，非直取部分网络成组
	大米	大米	18.00	21.48	车—库—船	卸车后网络成组
国外进口	杂货	集装箱	26.60	31.74	车—场—船	集装箱杂货
	橡胶	橡胶	5.00	5.97	船—船	直接过驳
	钢材	钢材	120.00	143.20	船—车；船—场—车	30%直接换装
	其它	集装箱	32.00	38.19	船—库—车	10%拆箱后汽车疏运

2.2 装卸工艺资料

表二中的工艺资料被整理为 process_options 表。每个可选工艺记录货类、操作过程、配机组合、配工人数、原始效率、效率单位、折算吨/小时效率、机械参与台数和备注。对于集装箱效率，原始单位为箱/小时，本文统一按 10 吨/箱折算为吨/小时，同时在技术标准图中保留原单位。

2.3 机械与人力资源条件

表三中的机械资源被整理为 machinery 表，包含机类、起重量、现有台数、司机人数和理论利用率。机械能力核算中，单台可用台时按营运天数、班制、每班小时、理论利用率以及 K 储、K 班折减后计算；司机人数作为机械使用的附加约束进入数学模型。港口装卸工人数为 1000 人，人力能力按计划期总可供工时核算。

2.4 货运量双口径校核

经校核，表一逐项合计为 419.0 万吨，而任务书正文要求计划期年货运量为 500 万吨。本文以 500 万吨为主控制口径，按货类原比例统一校正；同时保留表一原始 419 万吨口径，用于说明数据一致性修正对操作量、库场利用率和机械利用率的影响。

表 2-2 500 万吨口径与表一原始口径计算结果对比

计 算 口 径	吞 吐 量 (万 吨)	操 作 量 (万 吨)	操 作 系 数	直 接 换 装 比 重	仓 库 利 用 率	前 方 堆 场 利 用 率	后 方 堆 场 利 用 率	最 高 机 械 率	最 高 机 械 利 用 率	人 力 利 用 率
500 万	500.00	917.90	1.84	16.42%	155.08%	175.18%	30.00%	跨	358.20%	13.96%

吨校正									运	
口径									车	
表一原	419.00	769.20	1.84	16.42%	129.96%	146.80%	25.14%	跨	300.17%	11.70%
始419									运	
万吨口径									车	

2.5 计算参数与假设

全局参数包括泊位数 16 个，仓库面积 4.5 万平方米，前方堆场面积 3 万平方米，后方堆场面积 5 万平方米，n 班=3，K 出=0.85，K 利=0.7，μ=1/7，T 库=330 天，T 场=350 天，t 库=18 天，t 场=10 天，a 库=0.97T/m²，a 场=2.0T/m²。标准车辆载重按 60 吨/车折算，该参数为课程设计假设。

第 3 章 港口货流网络建模

3.1 货流网络节点与边

货流网络的节点包括船、车、库、场。船表示海船或驳船装卸端，车表示铁路或汽车集疏运端，库表示仓库及拆箱后入库环节，场表示前方或后方堆场。边表示装卸操作过程，例如车—场、场—船、船—车、船—库、库—车等。每条边记录货类、货量、是否直接换装、工艺方案、效率、配工人数、作业小时、作业工时和各机种机械台时。

3.2 货流路径拆分规则

含直接换装比例的货类按直接路径和中转路径拆分：矿石 50%船—车、50%船—场—车；国内进口杂货 25%船—车、75%船—库—车；海盐 50%车—船、50%车—场—船；钢材 30%船—车、70%船—场—车。生铁按电磁吸盘卸车 60%和常规成组卸车 40%拆分。其它集装箱严格按任务书路径船—库—车处理，其中 10%在库—车环节标记为拆箱后汽车疏运。

表 3-1 货流拆分结果摘录

flow_id	原始流向	货类	拆分比例	拆分后货量_万吨	路径	备注
F001	国内出口	杂货	1.00	41.77	车—场—船	集装箱杂货

F002	国内出口	生铁	0.60	64.44	车—场—船	电磁吸盘卸车 60%，后续成组装船
F003	国内出口	生铁	0.40	42.96	车—场—船	非电磁吸盘部分按成组货处理
F004	国内出口	水泥	1.00	17.90	车—库—船	卸车后全部货板
F005	国内出口	纯碱	1.00	22.67	车—场—船	成组
F006	国内进口	棉花	1.00	2.86	船—车	直接换装
F007	国内进口	矿石	0.50	21.48	船—车	直接换装
F008	国内进口	矿石	0.50	21.48	船—场—车	经堆场疏运
F009	国内进口	杂货	0.25	3.10	船—车	直接换装，不成组
F010	国内进口	杂货	0.75	9.31	船—库—车	入库部分全部货盘成组
F011	国外出口	海盐	0.50	5.73	车—船	直接换装
F012	国外出口	海盐	0.50	5.73	车—场—船	网络成组后装船
F013	国外出口	大米	1.00	21.48	车—库—船	卸车后网络成组
F014	国外出口	杂货	1.00	31.74	车—场—船	集装箱杂货
F015	国外进口	橡胶	1.00	5.97	船—船	直接过驳
F016	国外进口	钢材	0.30	42.96	船—车	直接换装
F017	国外进口	钢材	0.70	100.24	船—场—车	经堆场疏运
F018	国外进口	其它	0.90	34.37	船—库—车	任务书给定船—库—车，90%按整箱经库区疏运处理

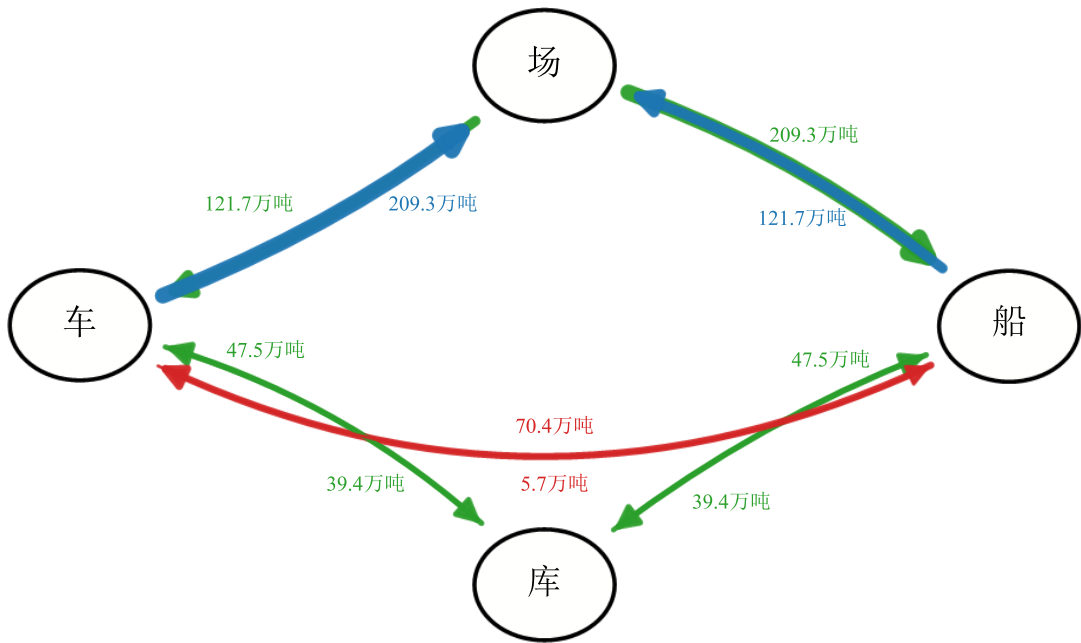
3.3 操作过程展开方法

路径展开时，将“车—场—船”拆为“车—场”和“场—船”，“船—场—车”拆为“船—场”和“场—车”，“船—库—车”拆为“船—库”和“库—车”。直接换装路径如“船—车”“车—船”“船—船”保留为单条直接边。展开后，吞吐量仍按原货流自然吨统计，操作量则按所有操作边吨数累加。

3.4 工艺方案匹配规则

每条操作边按“工艺货类+操作过程”匹配可选工艺。初始方案优先选择任务书中效率较高且专业化程度较高的工艺；若为生铁车—场边，则根据电磁吸盘比例选择对应工艺；若进行集装箱工艺情景或 MILP 优化，则在候选工艺集合内由模型选择。

3.5 货流网络模型图与说明



红色为直接换装，蓝色为入场路径，绿色为库场/常规连接；线宽按操作量缩放。

图 3-1 港口货流网络模型图

第 4 章 工艺选择优化模型与算法

4.1 问题抽象

本设计的数学模型分为两层。第一层为货流网络预处理：按照任务书给定的货类、流向、直接换装比例和特殊备注，确定拆分货流、固定路径量和操作边作业量。矿石、钢材、海盐和进口杂货等货类的直接换装比例作为任务书给定参数，不在 MILP 中优化。第二层为 Gurobi 工艺选择与设备增配 MILP：在每条操作边的可选工艺集合中选择一种工艺，并测算关键机种增配台数。该模型用于降低原始资源最大利用率、识别关键瓶颈并提出机械增配建议，不是完整的泊位调度、库场分配或直接换装比例优化模型。

因此，报告中的模型边界与程序保持一致：货流路径和直接换装比例在预处理阶段固定，Gurobi 求解层只优化操作边工艺选择变量 y_{ek} 和设备增配变量 z_m ；库场、泊

位和人力资源进入资源利用率评价，但库场分配、堆存期和直接换装比例不由该 MILP 重新决策。

4.2 决策变量

设 F 为拆分货流集合， R_f 为货流 f 的路径集合， E 为操作边集合， K_e 为操作边 e 的可选工艺集合， M 为机械集合。路径比例 ρ_{fr} 由任务书或备注确定， x_{fr} 和操作边量 G_e 均在预处理阶段计算得到，不作为 Gurobi 求解层的核心决策变量；Gurobi 模型中的决策变量为 y_{ek} 和 z_m ，其中 y_{ek} 为 0-1 工艺选择变量， z_m 为机种 m 的增配台数。 U 表示相对于原始资源能力的最大利用率上界， s_r 表示资源 r 按原始能力计的超载量。

表 4-1 模型符号说明

维度	符号	含义
集合	F, E, M	货流集合、操作边集合、机械集合
	R_f, K_e	货流 f 的路径集合、边 e 的可选工艺集合
参数	Q_f, ρ_{fr}, β_e	货流量、路径比例、直接换装比例
	v_{ek}, a_{ek}, b_{ekm}	工艺效率、配工人数、机种参与台数
预处理量	x_{fr}, G_e	路径作业量、操作边固定作业量
变量	y_{ek}, z_m, U, s_r	工艺选择变量、机种增配台数、原始最大资源利用率、资源超载量

4.3 目标函数

Gurobi 模型采用分层多目标而不是简单加权目标。第一优先级最小化原始资源能力下的最大利用率 U ，用于识别瓶颈强度；第二优先级最小化各资源原始超载比例之和；第三优先级最小化设备增配台数；第四优先级在前述目标不变的条件下减少总机械台时和总作业工时。需要说明的是， U 允许大于 1，因此模型不是简单要求所有原始资源利用率不超过 100%，而是在现有资源可能不足时寻找最小最大瓶颈并给出设备增配建议。

$$\text{lexmin} \left(U, \sum_{r \in R} \frac{s_r}{C_r}, \sum_{m \in M} z_m, \sum_{m \in M} H_m + \sum_{e \in E} L_e \right) \quad (1)$$

4.4 约束条件

约束条件同样分为两层。预处理层负责货流守恒、直接换装比例和操作边展开；Gurobi 求解层负责工艺唯一选择、机械能力、司机能力、泊位船边作业时间、固定库场负荷和人力能力。库场需求由预处理路径量决定，在 Gurobi 中作为固定资源负荷进入原始利用率评价，但本模型不优化库场分配、堆存期和直接换装比例。机械与司机资源通过 z_m 增配变量形成可执行能力；库场瓶颈仍通过提高直接换装比例、缩短堆存期和转移堆存区域等管理措施处理。

$$x_{fr} = \rho_{fr} Q_f, \sum_{r \in R_f} x_{fr} = Q_f \quad (2)$$

$$\sum_{f \in F_c} \sum_{r \in R_f^a} x_{fr} = \beta_c \sum_{f \in F_c} Q_f \quad (3)$$

$$\sum_{k \in K_e} y_{ek} = 1, y_{ek} \in \{0,1\} \quad (4)$$

$$H_m = \sum_{e \in E} \sum_{k \in K_e} b_{ekm} \frac{\hat{G}_e}{v_{ek}} y_{ek}, H_m \leq A_m U, H_m - A_m \leq s_m \quad (5)$$

$$D_m = \sum_{e \in E} \sum_{k \in K_e} d_{ekm} \frac{\hat{G}_e}{v_{ek}} y_{ek} \quad (6)$$

$$L_e = \sum_{k \in K_e} a_{ek} \frac{\hat{G}_e}{v_{ek}} y_{ek} \quad (7)$$

$$H_m \leq A_m + a_m z_m, D_m \leq B_m + b_m z_m \quad (8)$$

$$W_j \leq C_j U, W_j - C_j \leq s_j, C_j = A_j K_u a_j \frac{T_j}{t_j} \quad (9)$$

$$\sum_{e \in E_{\text{ship}}} T_e \leq C_{\text{berth}} U, C_{\text{berth}} = 16 T_y n h K_{\text{out}} \quad (10)$$

$$\sum_{e \in E} L_e \leq C_{\text{lab}} U, C_{\text{lab}} = N_{\text{worker}} T_y n h K_{\text{out}} \quad (11)$$

4.5 求解算法与求解器

表 4-2 求解算法比较与选择

算法	适用特点	本设计取舍
规则枚举汇总	按任务书比例和默认工艺逐边计算	作为初始方案，解释性强
Gurobi MILP	工艺选择、台时和工时可线性表达	作为本设计主优化方法
确定性贪心局部搜索	逐边尝试切换候选工艺，若字典序目标改善则接受	作为 Gurobi 的启发式对照算法
CP-SAT	适合复杂排程逻辑	缺少逐班次排程数据，暂不采用
GA/NSGA-II	适合非线性多目标黑箱搜索	本题规模小且线性结构明确，必要性不高
离散事件仿真	适合船车随机到达和排队	需补充到达分布与服务规则
强化学习	适合动态在线调度	课程设计数据不足，不作为本次方法

4.6 算法流程与结果校验

MILP 求解过程为：读取操作边、候选工艺、机械资源和资源能力表；为每个“操作边-工艺”组合建立二进制变量 y_{ek} ，为每个机种建立增配变量 z_m ；建立每条边只能选择一种工艺的等式约束；按工艺效率计算作业小时、机械台时、司机工时和作业工时；建立原始资源利用率约束和增配后机械/司机可执行约束；调用 Gurobi 求解；最后输出工艺选择、资源负荷、设备增配和模型求解日志。

表 4-3 Gurobi-MILP 求解流程中文伪代码

算法 1 给定货流路径下的工艺选择与设备增配 MILP 求解流程	
输入：candidate_options.csv, resource_capacities.csv, machinery.csv	
输出：工艺选择结果、资源负荷、设备增配量、模型文件与求解日志	
1:	初始化：读取候选工艺、资源能力和机械参数表。
2:	校验输入字段、候选工艺唯一性、资源容量正值和工时非负。
3:	对每条操作边 e 构造候选工艺集合 K_e
4:	建立二进制工艺选择变量和整数设备增配变量 $y_{ek} \in \{0,1\}, z_m \in \mathbb{Z}_+$
5:	建立工艺唯一选择约束 $\sum_{k \in K_e} y_{ek} = 1, \forall e \in E$
6:	按候选工艺效率计算作业小时、人工工时、机械台时和司机工时。 $T_{ek} = G_e / v_{ek}$
7:	汇总机械、司机、泊位、人力和固定库场资源需求。
8:	建立原始资源利用率上界和超载量约束。 $d_r \leq C_r U, d_r - C_r \leq s_r$
9:	对机械与司机资源建立增配后可执行约束。 $H_m \leq A_m + a_m z_m, D_m \leq B_m + b_m z_m$
10:	设置 Gurobi 分层多目标。 $lexmin(U, \sum_{r \in R} s_r / C_r, \sum_{m \in M} z_m, \sum_{m \in M} H_m + \sum_{e \in E} L_e)$
11:	调用 Gurobi 求解并导出模型文件、解文件和求解日志。
12:	输出工艺选择、资源负荷、设备增配量和模型摘要。
13:	运行确定性贪心局部搜索作为启发式对照。
14:	将 Gurobi 与启发式结果回填 Word、Excel 和图表数据。

表 4-4 Gurobi 模型规模与求解信息

项目	结果
求解器	Gurobi
status_code	2
solution_count	8
num_variables	104
num_constraints	132
objective_value	2.4540036484918586
max_original_resource_utilization_U	2.4540036484918586
total_added_equipment_units	41.0

mip_gap	NA
模型文件	process_selection_model.lp
解文件	process_selection_solution.sol
日志文件	gurobi.log

表 4-5 默认方案、贪心局部搜索与 Gurobi 方法对照表

方法	最大资源利用率	超载量	超载比例合计	总机械/人工小时	与 Gurobi 最大利用率差距
默认方案	358.20%	1,529,493	8.33	1,277,442	112.83%
贪心局部搜索	245.38%	1,478,997	5.78	1,368,673	0.00%
Gurobi MILP	245.38%	1,481,710	5.78	1,374,830	0.00%

方法对照结果显示，确定性贪心局部搜索在最大资源利用率和超载比例合计上达到与 Gurobi 接近的水平，说明候选工艺集合中存在比较明确的瓶颈转移方向；但 Gurobi 进一步考虑设备增配变量和机械/司机可执行约束，因此总机械/人工小时不一定最低。本文以 Gurobi 作为正式优化方案，主要原因在于其能够同时输出工艺选择、机种增配台数和增配后资源利用率。

确定性贪心局部搜索结果已作为 Gurobi 模型的对照算法纳入分析。该算法从默认工艺出发，逐边尝试切换候选工艺，当最大资源利用率、超载比例和总机械/人工小时构成的字典序目标改善时接受新方案。输出结果显示，算法迭代 11 轮后，最大资源利用率为 245.38%，超载比例合计为 5.78，总机械/人工小时为 1368673 小时。该结果主要用于验证工艺切换对资源负荷的改善方向，不替代 Gurobi 中设备增配变量 z_m 形成的正式优化方案。

默认方案、贪心局部搜索与 Gurobi 结果对比

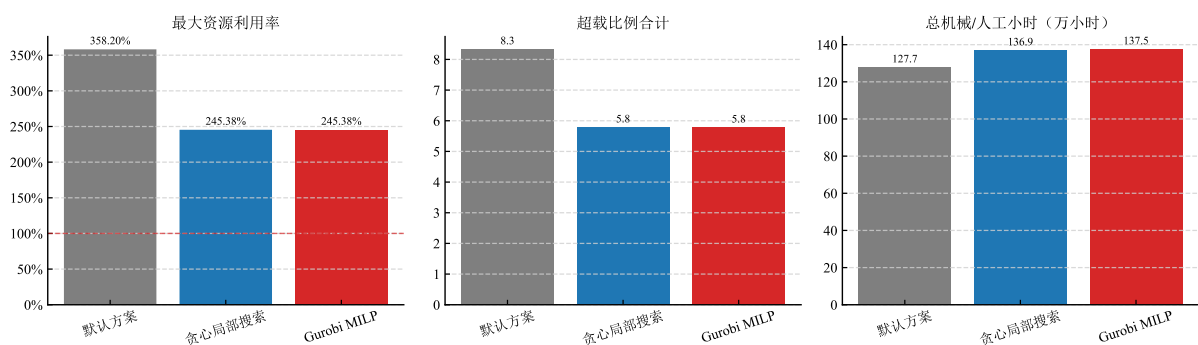


图 4-1 默认方案、贪心局部搜索与 Gurobi 结果对比图

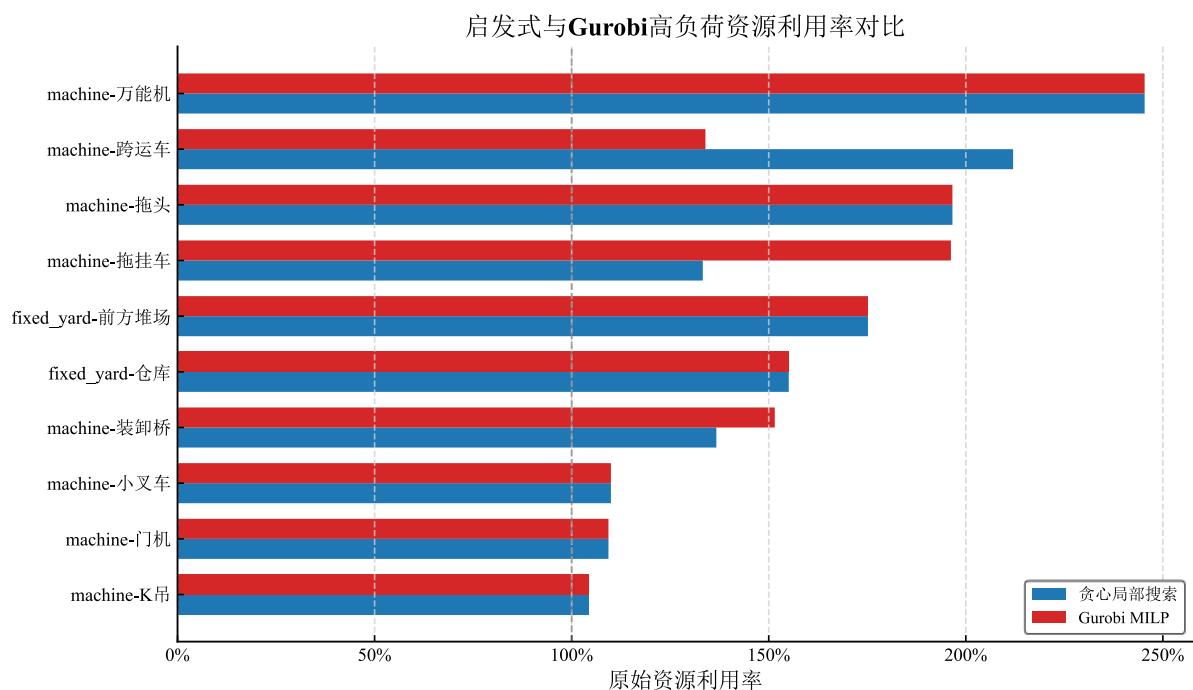


图 4-2 启发式与 Gurobi 高负荷资源利用率对比图

由图 4-2 可以看出，贪心局部搜索与 Gurobi 均识别出万能机、拖头、拖挂车、前方堆场和仓库等高负荷资源。二者在个别机种上的利用率排序存在差异，说明逐边贪心切换能够提供较好的可行改进方向，但缺少全局整数规划对设备增配和资源约束的统一协调。因此，正文后续仍以 Gurobi MILP 作为工艺选择与设备增配的正式模型，以启发式结果作为算法稳健性和瓶颈识别一致性的旁证。

第 5 章 装卸作业技术方案编制

5.1 初始方案

初始方案按默认工艺规则生成，作为能力核算和瓶颈识别的基准方案。需要说明的是，初始方案并非已经完全满足所有资源约束的最终可执行方案，而是用于揭示在现有路径比例和默认工艺下哪些资源压力最大。

正文表格只列示关键操作边摘要，完整 32 条操作过程、各机种机械作业量和机械台时见 Excel “装卸作业技术方案主表”和“操作过程明细表”。

表 5-1 初始方案操作过程明细摘录

流向	货类	操作过程	操作量_万	配机	效率(t/h)	作业小
----	----	------	-------	----	---------	-----

			吨			时
国内出口	杂货	车—场	41.77	轮胎龙门吊	200.00	2088.31
国内出口	杂货	场—船	41.77	跨运车—装卸桥	200.00	2088.31
国内出口	生铁	车—场	64.44	桥吊—电磁吸盘	100.00	6443.91
国内出口	生铁	场—船	64.44	K 吊—牵引车—门机	80.00	8054.89
国内出口	生铁	车—场	42.96	人力—万能机	50.00	8591.89
国内出口	生铁	场—船	42.96	K 吊—牵引车—门机	80.00	5369.93
国内出口	水泥	车—库	17.90	小叉车	35.00	5114.22
国内出口	水泥	库—船	17.90	小叉车—万能机—门机	40.00	4474.94
国内出口	纯碱	车—场	22.67	万能机	60.00	3778.84
国内出口	纯碱	场—船	22.67	K 吊—牵引车—门机	50.00	4534.61

5.2 MILP 优化方案

独立 Gurobi 模型已在 53 个候选工艺中完成工艺选择与设备增配求解，原始资源最大利用率 U 为 245.40%，设备增配总台数为 41 台。Gurobi 结果同时输出原始资源利用率、增配后资源利用率和机种增配台数。需要说明的是，库场和泊位作为固定资源负荷纳入评价，工艺选择不能改变入库、入场货量。

表 5-2 Gurobi 工艺选择结果摘录

边 ID	货类	操作过程	配机	效率(t/h)	作业小时	工时	机械台时	默认工艺
E0001	杂货	车—场	集装箱正面吊	150.00	2784.41	5568.81	2784.41	False
E0002	杂货	场—船	跨运车—装卸桥	200.00	2088.31	14,618	6264.92	True
E0003	生铁	车—场	桥吊—电磁吸盘	100.00	6443.91	19,332	6443.91	True
E0004	生铁	场—船	K 吊—牵引车—门机	80.00	8054.89	72,494	32,220	True
E0005	生铁	车—场	人力—万能机	50.00	8591.89	51,551	8591.89	True
E0006	生铁	场—船	K 吊—牵引车—门机	80.00	5369.93	48,329	21,480	True
E0007	水泥	车—库	小叉车	35.00	5114.22	40,914	5114.22	True
E0008	水泥	库—船	小叉车—船机	35.00	5114.22	92,056	10,228	False
E0009	纯碱	车—场	万能机	60.00	3778.84	15,115	3778.84	True
E0010	纯碱	场—船	柴吊—门机	50.00	4534.61	40,811	13,604	False

5.3 分货类工艺方案

表 5-3 分货类装卸工艺方案摘录

货类	操作过程	选定配机	原效率	原单位	折算效率(t/h)	配工人数	备注
----	------	------	-----	-----	-----------	------	----

杂货	车—场	轮胎龙门吊	20	箱/h	200.00	2	最大码高三层
杂货	场—船	跨运车—装卸桥	20	箱/h	200.00	7	2 台跨运车
生铁	车—场	桥吊—电磁吸盘	100	T/h	100.00	3	
生铁	场—船	K 吊—牵引车—门机	80	T/h	80.00	9	牵引托盘 2 组
生铁	车—场	人力—万能机	50	T/h	50.00	6	
水泥	车—库	小叉车	35	T/h	35.00	8	
水泥	库—船	小叉车—万能机—门机	40	T/h	40.00	15	小叉车/万能机各 2 台
纯碱	车—场	万能机	60	T/h	60.00	4	
纯碱	场—船	K 吊—牵引车—门机	50	T/h	50.00	9	牵引车 2 组
棉花	船—车	门机	30	T/h	30.00	13	

5.4 技术操作标准

典型货类装卸工艺流程

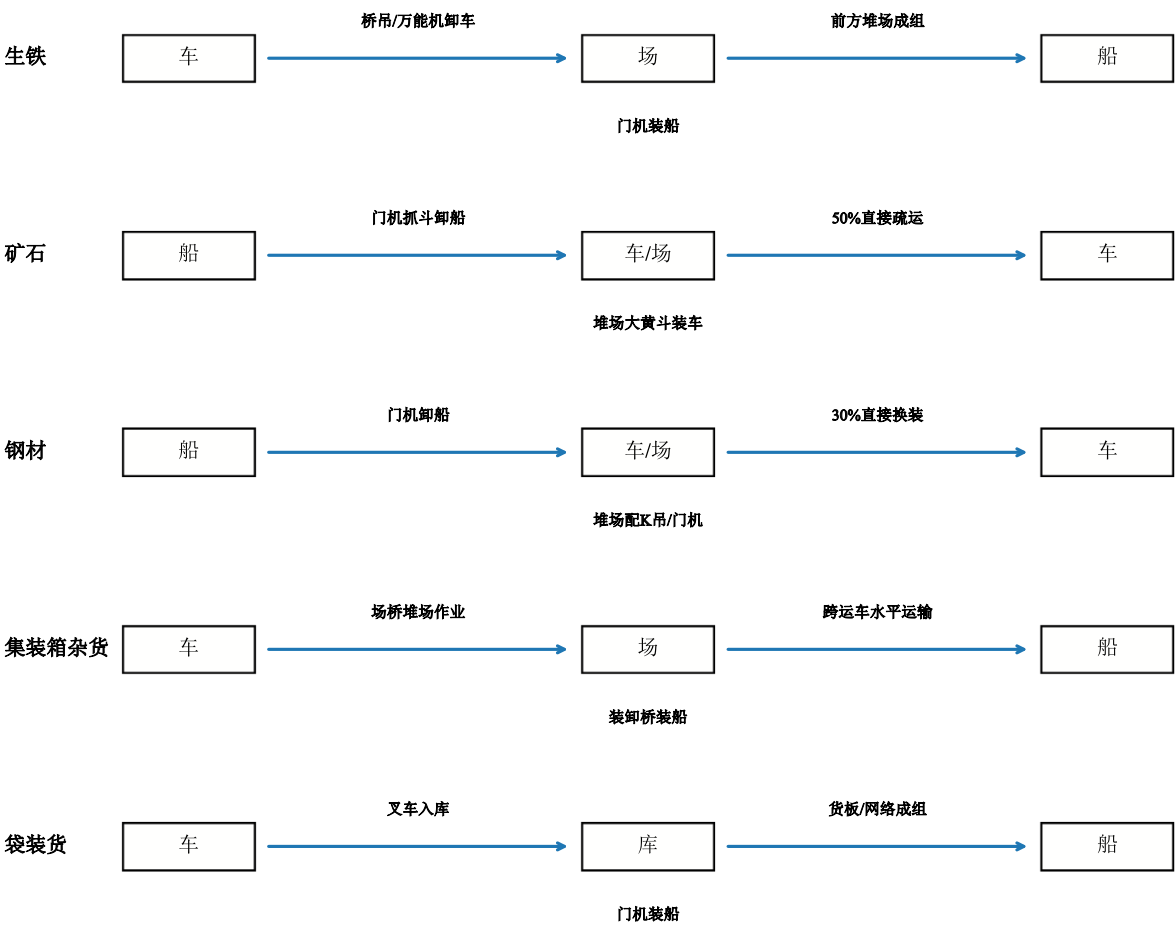


图 5-1 典型货类装卸工艺流程图

表 5-4 典型装卸技术操作标准表

货类	操作过程	选定配机	原效率	折算效率 (t/h)	配工人数	技术要点
杂货	车—场	轮胎龙门吊	20 箱/h	200	2	最大码高三层
杂货	场—船	跨运车—装卸桥	20 箱/h	200	7	2 台跨运车
生铁	车—场	桥吊—电磁吸盘	100T/h	100	3	按成组/机械化连续作业组织
生铁	场—船	K 吊—牵引车—门机	80T/h	80	9	牵引托盘 2 组
生铁	车—场	人力—万能机	50T/h	50	6	按成组/机械化连续作业组织
水泥	车—库	小叉车	35T/h	35	8	按成组/机械化连续作业组织
水泥	库—船	小叉车—万能机—门机	40T/h	40	15	小叉车/万能机各 2 台
矿石	船—车	门机—抓斗	65T/h	65	5	按成组/机械化连续作业组织
矿石	船—场	门机—抓斗	70T/h	70	5	推扒机清仓量占 20%
矿石	场—车	大黄斗	150T/h	150	2	按成组/机械化连续作业组织
杂货	船—车	门机	25T/h	25	13	按成组/机械化连续作业组织
杂货	船—库	门机—牵引车—小叉车	35T/h	35	13	牵引车 2 组
杂货	库—车	小叉车	25T/h	25	6	按成组/机械化连续作业组织
海盐	车—船	门机	50T/h	50	13	按成组/机械化连续作业组织

5.5 方案比较

初始方案突出效率优先和专业化机械优先，便于组织连续作业；MILP 方案则更强调资源负荷均衡，能够减少部分瓶颈机种的台时占用。两者比较说明，在课程设计给定资源条件下，单纯提高过程效率不一定能形成最均衡的生产组织，必须结合机械台数、司机人数和库场能力共同判断。

第 6 章 主要指标计算

6.1 吞吐量、操作量与操作系数

$$Q = \sum_f Q_f = 500.00 \times 10^4 t \quad (12)$$

$$G = \sum_e G_e = 917.90 \times 10^4 t \quad (13)$$

$$K_{op} = G / Q = 917.90 / 500.00 = 1.836 \quad (14)$$

吞吐量按计划期自然吨统计，不因货物进入库场而重复。操作量按所有操作边作业吨累加，反映港内装卸搬运的实际工作量。操作系数大于 1，说明相当部分货物需要经过库场中转或二次操作。

6.2 直接换装与机械化作业比重

$$R_d = Q_d / Q = 82.10 / 500.00 = 16.42\% \quad (15)$$

$$R_m = G_m / G = 100.00\% \quad (16)$$

直接换装包括船—车、车—船和船—船等不经过库场的直接作业。机械化作业比重在本文中采用“机械主导作业比重”口径，即主要起落、搬运由机械完成的工序吨占总工序吨；人工参与挂钩、摘钩、指挥和辅助整理，不计为非机械化。另需说明，分机种机械起运量是按机种累计统计，同一操作边若同时使用门机、牵引车和叉车，会在多个机种下分别形成机械起运量，因此分机种机械起运量合计可以大于总操作量。

6.3 同时开头量

$$N_{ship} = \frac{\sum_{e \in E_{ship}} T_e}{T_y n 8 K_{out}} K_{st} = 20 \quad (17)$$

$$N_{veh} = \frac{\sum_{e \in E_{veh}} T_e}{T_y n 8 K_{out}} K_{st} = 19 \quad (18)$$

N 舱表示同时需要组织的船边舱口或作业头数量，不等同于泊位数量；N 车表示车辆相关作业头数量。两者均由作业小时折算得到，用于反映计划期生产组织强度。

6.4 机械台时、机种需要量与利用率

$$T_e = G_e / v_e \quad (19)$$

$$H_m = \sum_{e \in E} b_{em} T_e \quad (20)$$

$$N_m^{need} = H_m / A_m^{single} \quad (21)$$

$$u_m = H_m / A_m \quad (22)$$

表 6-1 机械能力与利用率统计表

机类	现有台数	计划期所需台时	单台可用台时	需要台数	机械利用率	平均台时效率_t_h	是否短缺
门机	25	89,780	2227.29	40.31	161.24%	54.42	是
桥吊	3	6443.91	2227.29	2.89	96.44%	100.00	否
装卸桥	2	5584.73	2227.29	2.51	125.37%	200.00	是
场桥	4	5393.79	2004.56	2.69	67.27%	200.00	否
K 吊	20	19,494	1781.83	10.94	54.70%	72.60	否
柴吊	8	0.00	1781.83	0.00	0.00%	0.00	否
正面吊	2	0.00	1781.83	0.00	0.00%	0.00	否
跨运车	2	11,169	1559.10	7.16	358.20%	100.00	是
拖挂车	4	0.00	1781.83	0.00	0.00%	0.00	否
万能机	12	61,416	1781.83	34.47	287.23%	29.92	是
大叉车	19	0.00	1781.83	0.00	0.00%	0.00	否
小叉车	15	32,884	1781.83	18.46	123.04%	30.77	是
大黄斗	2	1431.98	1781.83	0.80	40.18%	150.00	否
拖头	10	44,102	1781.83	24.75	247.51%	32.90	是
推扒机	2	3068.53	1781.83	1.72	86.11%	14.00	否

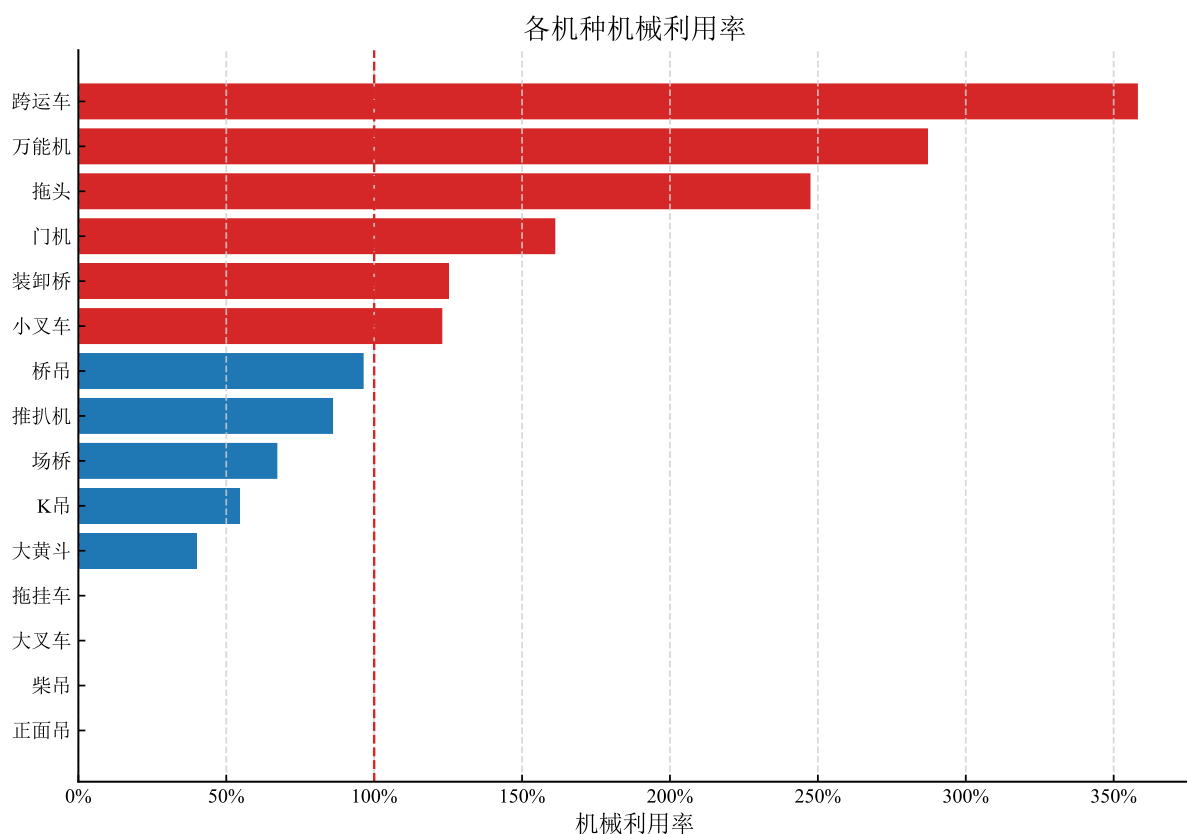


图 6-1 各机种机械利用率图

6.5 工时效率与劳动生产率

$$P_{syn} = G / L = 9.21t / \text{labor hour} \quad (23)$$

$$P_{ship} = G_{ship} / L_{ship} = 7.09t / \text{labor hour} \quad (24)$$

$$P_{veh} = G_{veh} / L_{veh} = 11.59t / \text{labor hour} \quad (25)$$

$$P_{lab} = Q / N_{worker} = 5000.00t / \text{person} \quad (26)$$

工时效率采用操作量除以相应作业工时，P 船和 P 车分别只统计船舶相关边和车辆相关边。劳动生产率采用吞吐量/装卸工人数作为主口径，并以操作量/装卸工人数作为辅助口径。

6.6 平均舱时量、车时量和车辆数

$$\bar{P}_{hatch} = 63.57t / h \quad (27)$$

$$\bar{P}_{veh} = 64.12t / h \quad (28)$$

$$N_{load} = 39936veh, N_{unload} = 42403veh (60t / veh) \quad (29)$$

表 6-2 主要生产指标汇总表

指标名称	单位	数值	说明
吞吐量	万吨	500.00	按计划期自然吨统计
操作量	万吨	917.90	所有操作过程作业量之和
机械起运量	万吨	1,481.62	按机种机械作业量累加
系数	次/吨	1.836	操作量/吞吐量
直接换装比重	%	16.42%	直接换装货量/总吞吐量
机械化作业比重	%	100.00%	机械化工序吨/总工序吨
同时开头量 N 舱	个	20	平均开工舱口数乘不平衡系数后取整
同时开头量 N 车	辆	19	平均车作业头数乘不平衡系数后取整
P 综	吨/工时	9.21	总操作量/总作业工时
P 船	吨/工时	7.09	船舶相关操作量/相关作业工时
P 车	吨/工时	11.59	车辆相关操作量/相关作业工时
劳动生产率(吞吐量)	吨/人	5,000.00	吞吐量/装卸工人数
劳动生产率(操作量)	吨/人	9,179.00	操作量/装卸工人数
平均舱时量	吨/小时	63.57	船舶相关操作量/船舶相关作业小时
平均车时量	吨/小时	64.12	车辆相关操作量/车辆相关作业小时
装车数	车	39,936	车辆到达边操作量/60 吨标准车
卸车数	车	42,403	车辆发出边操作量/60 吨标准车

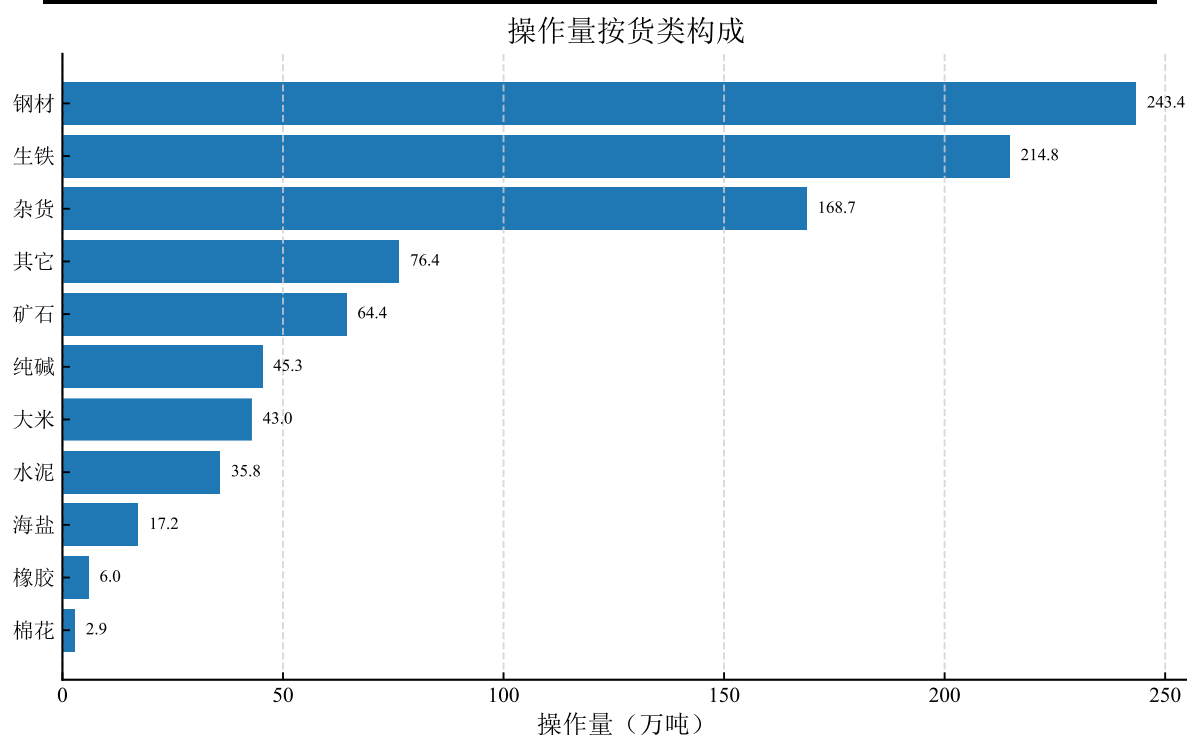


图 6-2 操作量按货类构成图

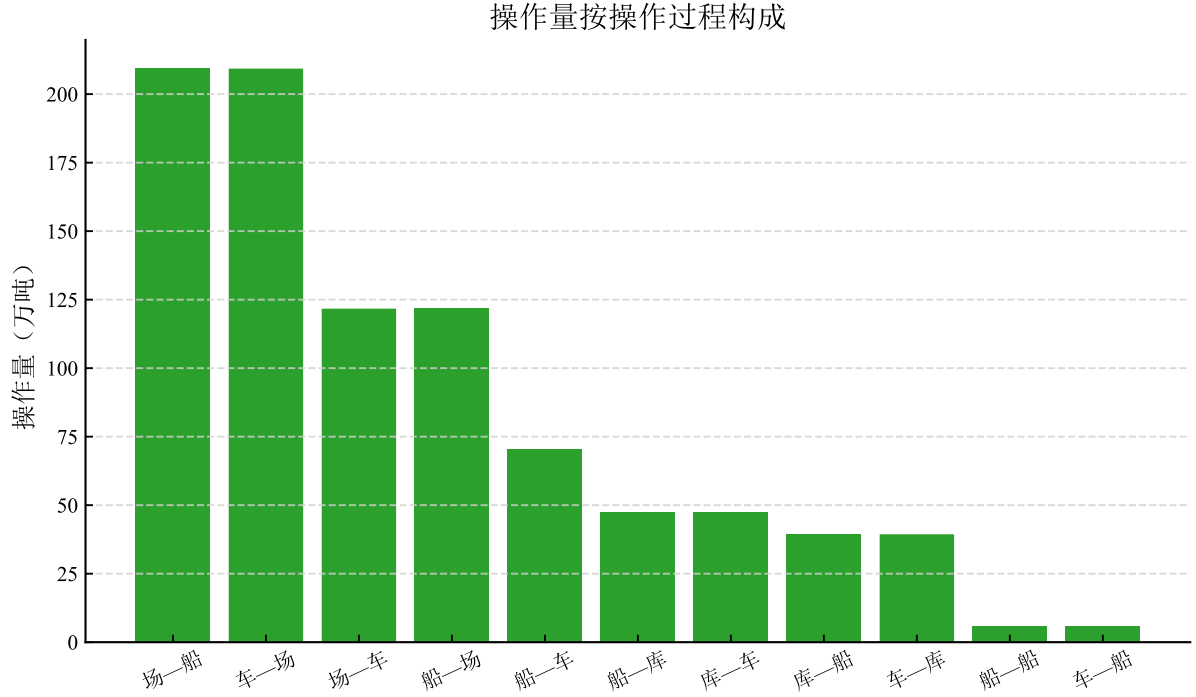


图 6-3 操作量按操作过程构成图

第 7 章 通过能力核算与薄弱环节分析

7.1 泊位通过能力

$$P_{berth} = 16T_y 24p_{line} K_u (1 - \mu) \quad (30)$$

泊位能力按 16 个泊位、营运天数、船舶相关操作边加权平均效率和非生产时间折减估算。泊位约束在 MILP 中以船边总作业时间形式表示，从而与船边主机效率和作业小时联动。

7.2 仓库与堆场通过能力

$$C_j = A_j K_u a_j (T_j / t_j), j \in \text{warehouse, front, rear} \quad (31)$$

库场能力按面积、面积利用系数、单位面积堆存定额和周转次数计算。直接换装货物不占用库场；入库边计入仓库需求，入场边按堆场分区计入前方或后方堆场需求。

7.3 机械与人力通过能力

$$A_m = N_m T_y n 8 \eta_m / (K_{st} K_{sh}) \quad (32)$$

$$\sum_{e \in E} L_e \leq C_{lab} U, C_{lab} = N_{worker} T_y n h K_{out} . \quad (33)$$

人力能力公式采用 T 场、 n 班和 K 出，与表格口径保持一致；不再使用漏掉班制的简化式。机械能力除台数外，还在模型中引入司机人数约束，用于反映表三中司机配置的资源含义。

表 7-1 泊位、库场与人力通过能力核算表

环节	需求量	可供能力	利用率	计算口径
泊位系统	5,000,000	5,125,950	97.54%	$16 \text{ 个泊位} \times \text{营运天数} \times \text{昼夜小时} \times \text{船边平均作业线效率} \times K \text{ 利} \times (1-\mu)$
仓库	868,735	560,175	155.08%	$A \text{ 库} \times a \text{ 库} \times K \text{ 利} \times T \text{ 库} / t \text{ 库}$
前方堆场	2,575,179	1,470,000	175.18%	$A \text{ 场前} \times a \text{ 场} \times K \text{ 利} \times T \text{ 场} / t \text{ 场}$
后方堆场	735,084	2,450,000	30.00%	$A \text{ 场后} \times a \text{ 场} \times K \text{ 利} \times T \text{ 场} / t \text{ 场}$
装卸工人	996,674	7,140,000	13.96%	$1000 \text{ 人} \times T \text{ 场} \times n \text{ 班} \times 8 \text{ 小时} \times K \text{ 出}$

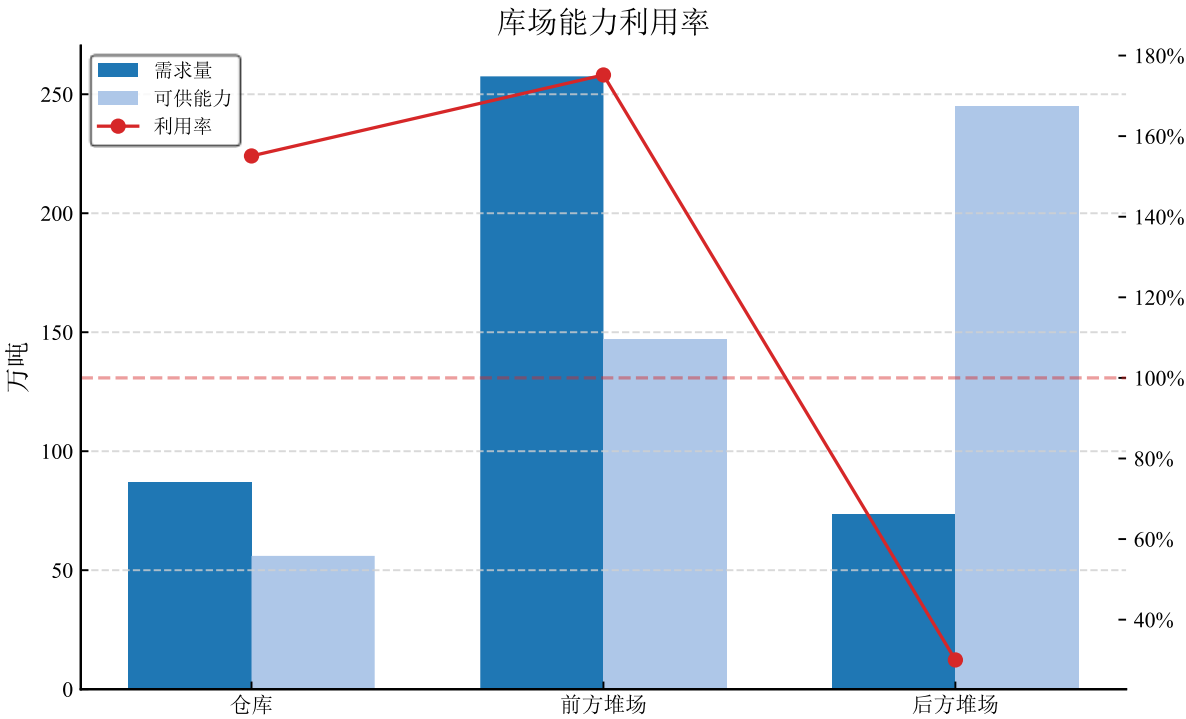


图 7-1 库场能力利用率图

7.4 集疏运能力

集疏运能力以车辆相关操作量、装车数、卸车数和车时量作为辅助判断。由于任务书未给出车辆到达分布和线路能力，本文不构造独立排队模型，而是将车相关作业小时和车辆折算数用于判断车辆组织强度。

7.5 薄弱环节识别

资源利用率=需求量/可供能力。利用率超过 1 表示该环节在初始方案下已经不足，接近 1 表示存在临界风险。

表 7-2 薄弱环节识别表

资源	需求量	可供能力	资源利用率	是否短缺	薄弱程度
机械-跨运车	11,169	3118.21	358.20%	是	不足
机械-万能机	61,416	21,382	287.23%	是	不足
机械-拖头	44,102	17,818	247.51%	是	不足
库场-前方堆场	2,575,179	1,470,000	175.18%		不足
机械-门机	89,780	55,682	161.24%	是	不足
库场-仓库	868,735	560,175	155.08%		不足
机械-装卸桥	5584.73	4454.58	125.37%	是	不足
机械-小叉车	32,884	26,727	123.04%	是	不足
泊位-泊位系统	5,000,000	5,125,950	97.54%		临界
机械-桥吊	6443.91	6681.87	96.44%	否	临界
机械-推扒机	3068.53	3563.66	86.11%	否	偏高
机械-场桥	5393.79	8018.24	67.27%	否	宽松

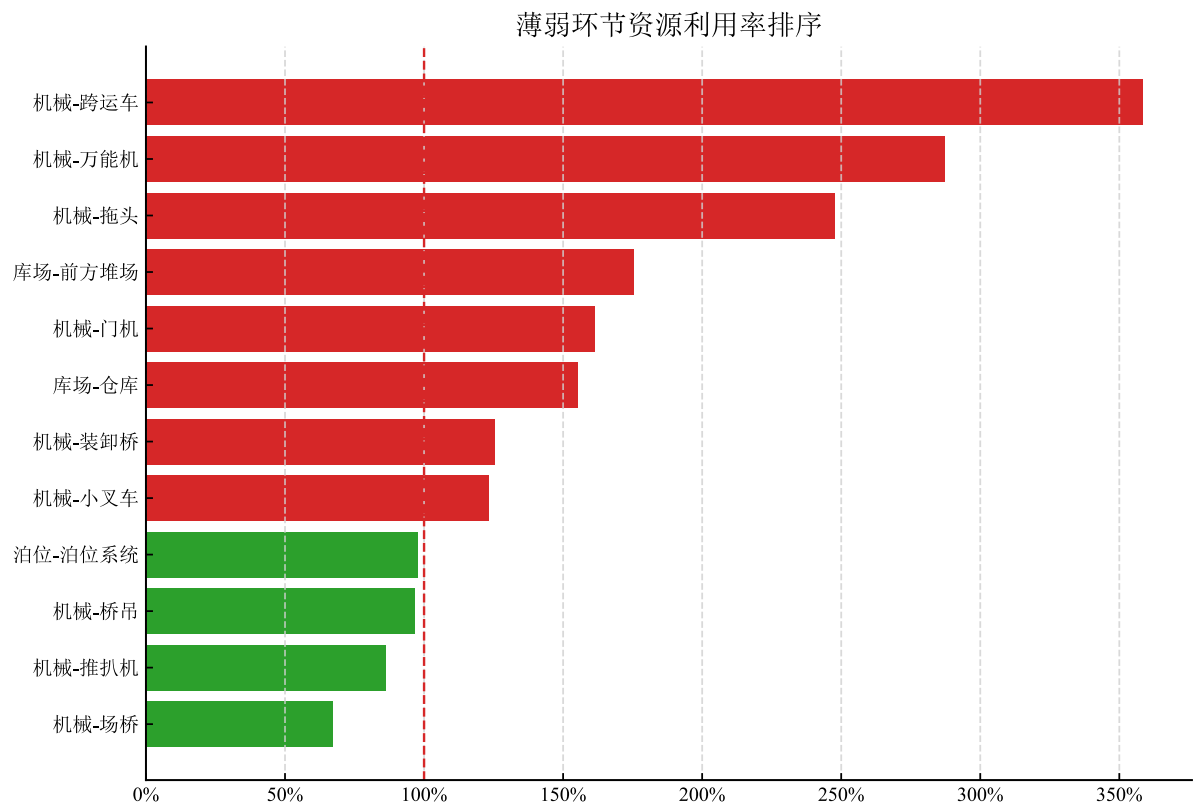


图 7-2 薄弱环节资源利用率排序图

第 8 章 情景分析与资源改进建议

8.1 情景设置

本文设置三类情景：情景 A 将具备直接换装条件的货类直接换装比例提高 10 个百分点；情景 B 将 t 库由 18 天降至 16 天、t 场由 10 天降至 8 天；情景 C 将部分集装箱船边水平运输由跨运车工艺切换为龙门—牵引拖挂—装卸桥工艺。另将 MILP 工艺选择优化结果作为对照。

8.2 情景结果比较

表 8-1 初始方案与优化情景对比表

情景	操作系数	直接换装比重	仓库利用率	前方堆场利用率	后方堆场利用率	最高机械利用率	总作业工时	总机械台时
初始方案	1.84	16.42%	155.08%	175.18%	30.00%	358.20%	996,674	280,768
情景 A：直接换装+10pct	1.79	20.62%	152.87%	161.74%	30.00%	358.20%	990,844	270,305
情景 B：堆存期缩短	1.84	16.42%	137.85%	140.15%	24.00%	358.20%	996,674	280,768
情景 C：集装箱拖挂工艺	1.84	16.42%	155.08%	175.18%	30.00%	313.43%	1,009,705	301,246
MILP：工艺选择优化	1.84	16.42%	155.08%	175.18%	30.00%	245.38%	1,054,195	258,117

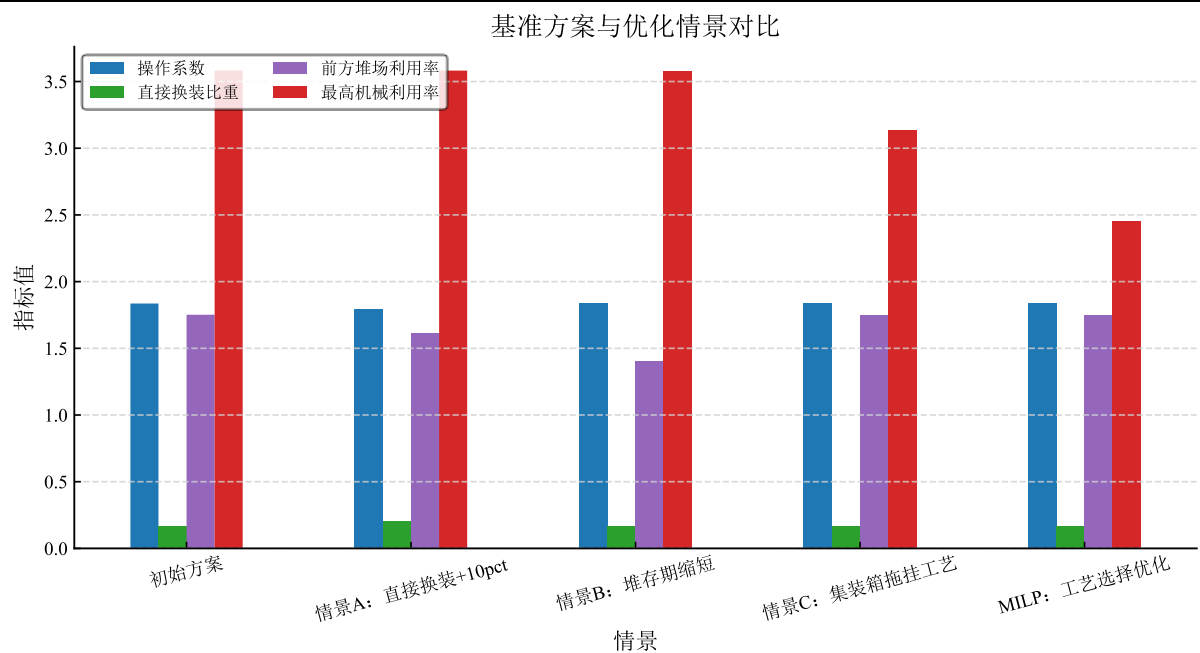


图 8-1 初始方案与优化情景对比图

各情景指标结果见表 8-1 和图 8-1。

需要说明的是，表 8-1 中的最高机械利用率按原始设备能力计算，用于衡量原有条件下的瓶颈强度；后文表 8-3 中的增配后利用率则按 Gurobi 增配变量 z_m 形成的新能力计算，用于判断增配方案的可执行性。

情景 C 能够转移跨运车负荷，但最高机械利用率仍为 313.43%，说明该情景只能缓解局部瓶颈，不能单独消除机械能力不足。MILP 方案同样是在现有候选工艺范围内的负荷均衡结果，不应表述为已完全可行的最终方案。

8.3 改进措施建议

表 8-2 主要薄弱环节改进措施

薄弱环节	主要原因	改进措施
跨运车 万能机	集装箱船边水平运输集中 件杂货和袋装货中转工艺占用多	切换部分作业为龙门—牵引拖挂—装卸桥，或增配跨运车 部分钢材、生铁和袋装货改用叉车、K 吊、门机或船机配合
拖头/拖挂车 前方堆场 门机	牵引组在成组作业和集装箱替代工艺中集中占用 非直接换装货物入场量大、堆存期较长 船边主机承担多货类装卸	优化拖挂配置，减少同时高峰，必要时临时租用 提高直接换装比例，压缩 t 场，部分货类转后方堆场 错峰安排货类，优先保障船边作业，部分环节采用船吊替代

表 8-3 关键资源缺口与 Gurobi 增配结果表

资源	初始缺口	z_m 输出	增配后台数	初始利用率	增配后利用率
机械-跨运车	6 台	1	3.00	358.20%	89.30%
机械-万能机	23 台	18	30.00	287.23%	98.15%
机械-拖头	15 台	10	20.00	247.51%	98.30%
机械-门机	16 台	3	28.00	161.24%	97.62%
机械-装卸桥	1 台	2	4.00	125.37%	75.77%
机械-小叉车	4 台	2	17.00	123.04%	97.02%
库场-仓库	30.86 万吨	本 MILP 不扩容库场	不适用	155.08%	不适用
库场-前方堆场	110.52 万吨	本 MILP 不扩容库场	不适用	175.18%	不适用

表 8-3 列出初始方案按能力核算反推的资源缺口，以及 Gurobi 模型中设备增配变量 z_m 的输出结果。Gurobi 增配变量主要用于机械和司机资源，库场和泊位瓶颈只参与

利用率评价，仍需通过提高直接换装比例、缩短堆存期、转移堆存区域或扩容等管理措施处理。

结 论

本文基于货流网络模型完成某港计划期装卸作业技术方案编制。按 500 万吨主口径计算，初始方案吞吐量为 500.00 万吨，总操作量为 917.9 万吨，操作系数为 1.836，直接换装比重为 16.42%，机械主导作业比重为 100.00%。

通过能力核算表明，初始方案能够形成完整的装卸作业组织框架，但库场和部分机械存在高负荷甚至能力不足风险。提高直接换装比例、缩短堆存期和调整部分工艺可以缓解压力，但在现有资源条件下仍需增配关键机械、优化堆场周转并加强车船衔接组织。

Gurobi 增配结果显示，关键机械增配重点集中在万能机增配 18 台、拖头增配 10 台、拖挂车增配 4 台、门机增配 3 台、小叉车增配 2 台、装卸桥增配 2 台、K 吊增配 1 台、跨运车增配 1 台，其中万能机、拖头和门机是主要调整对象。这一结果与薄弱环节排序相互印证，说明机械系统改进应优先围绕件杂货中转、牵引运输和船边主机组织展开。

本设计仍受任务书资料限制，船期、车辆到达分布、泊位专业化分工和不同作业线实际效率未进一步细化。后续可在本模型基础上引入离散事件仿真或分班次调度模型，使通过能力核算更贴近生产现场。

参考文献

- [1] 大连海事大学. 《港口管理》课程设计任务书[R]. 大连: 大连海事大学, 2026.
- [2] 国家质量监督检验检疫总局, 中国国家标准化管理委员会. 信息与文献 参考文献著录规则: GB/T 7714-2015[S]. 北京: 中国标准出版社, 2015.
- [3] 中华人民共和国交通运输部. 海港总体设计规范: JTS 165-2013[S]. 北京: 人民交通出版社, 2013.
- [4] 中华人民共和国交通运输部. 海港集装箱码头设计规范: JTS 165-4-2011[S]. 北京: 人民交通出版社, 2011.
- [5] 中华人民共和国交通运输部. 港口工程荷载规范: JTS 144-1-2010[S]. 北京: 人民交通出版社, 2010.
- [6] 中华人民共和国交通运输部. 码头结构设计规范: JTS 167-2018[S]. 北京: 人民交通出版社, 2018.
- [7] 中华人民共和国交通运输部. 水运工程设计通则: JTS 141-2011[S]. 北京: 人民交通出版社, 2011.
- [8] 大连海事大学. 博士、硕士学位论文格式规范[Z]. 大连: 大连海事大学, 2026.
- [9] UNCTAD. Review of Maritime Transport 2024[R]. Geneva: United Nations, 2024.
- [10] The World Bank. Port Reform Toolkit[M]. 2nd ed. Washington, DC: The World Bank, 2007.
- [11] Notteboom T, Pallis A, Rodrigue J P. Port Economics, Management and Policy[M]. New York: Routledge, 2022.
- [12] Stopford M. Maritime Economics[M]. 3rd ed. London: Routledge, 2009.
- [13] Talley W K. Port Economics[M]. London: Routledge, 2009.
- [14] Rodrigue J P. The Geography of Transport Systems[M]. 5th ed. New York: Routledge, 2020.
- [15] Notteboom T E, Rodrigue J P. Port regionalization: towards a new phase in port development[J]. Maritime Policy & Management, 2005, 32(3): 297-313.
- [16] Wang T F, Cullinane K. The efficiency of European container terminals and implications for supply chain management[J]. Maritime Economics & Logistics, 2006, 8(1): 82-99.

- [17] Tongzon J. Efficiency measurement of selected Australian and other international ports using data envelopment analysis[J]. *Transportation Research Part A: Policy and Practice*, 2001, 35(2): 113-128.
- [18] Cullinane K, Wang T F, Song D W, Ji P. The technical efficiency of container ports: comparing data envelopment analysis and stochastic frontier analysis[J]. *Transportation Research Part A: Policy and Practice*, 2006, 40(4): 354-374.
- [19] Roll Y, Hayuth Y. Port performance comparison applying data envelopment analysis[J]. *Maritime Policy & Management*, 1993, 20(2): 153-161.
- [20] Steenken D, Voss S, Stahlbock R. Container terminal operation and operations research: a classification and literature review[J]. *OR Spectrum*, 2004, 26(1): 3-49.
- [21] Stahlbock R, Voss S. Operations research at container terminals: a literature update[J]. *OR Spectrum*, 2008, 30(1): 1-52.
- [22] Vis I F A, de Koster R. Transshipment of containers at a container terminal: an overview[J]. *European Journal of Operational Research*, 2003, 147(1): 1-16.
- [23] Bierwirth C, Meisel F. A survey of berth allocation and quay crane scheduling problems in container terminals[J]. *European Journal of Operational Research*, 2010, 202(3): 615-627.
- [24] Bierwirth C, Meisel F. A follow-up survey of berth allocation and quay crane scheduling problems in container terminals[J]. *European Journal of Operational Research*, 2015, 244(3): 675-689.
- [25] Carlo H J, Vis I F A, Roodbergen K J. Storage yard operations in container terminals: literature overview, trends, and research directions[J]. *European Journal of Operational Research*, 2014, 235(2): 412-430.
- [26] Carlo H J, Vis I F A, Roodbergen K J. Transport operations in container terminals: literature overview, trends, and research directions[J]. *European Journal of Operational Research*, 2014, 236(1): 1-13.
- [27] Imai A, Nishimura E, Papadimitriou S. The dynamic berth allocation problem for a container port[J]. *Transportation Research Part B: Methodological*, 2001, 35(4): 401-417.
- [28] Imai A, Nishimura E, Papadimitriou S. Berth allocation with service priority[J]. *Transportation Research Part B: Methodological*, 2003, 37(5): 437-457.
- [29] Cordeau J F, Laporte G, Legato P, Moccia L. Models and tabu search heuristics for the berth-allocation problem[J]. *Transportation Science*, 2005, 39(4): 526-538.

- [30] Kim K H, Moon K C. Berth scheduling by simulated annealing[J]. *Transportation Research Part B: Methodological*, 2003, 37(6): 541-560.
- [31] Park Y M, Kim K H. A scheduling method for berth and quay cranes[J]. *OR Spectrum*, 2003, 25(1): 1-23.
- [32] Meisel F, Bierwirth C. Heuristics for the integration of crane productivity in the berth allocation problem[J]. *Transportation Research Part E: Logistics and Transportation Review*, 2009, 45(1): 196-209.
- [33] Iris C, Pacino D, Ropke S, Larsen A. Integrated berth allocation and quay crane assignment problem: set partitioning models and computational results[J]. *Transportation Research Part E: Logistics and Transportation Review*, 2015, 81: 75-97.
- [34] Umang N, Bierlaire M, Vacca I. Exact and heuristic methods to solve the berth allocation problem in bulk ports[J]. *Transportation Research Part E: Logistics and Transportation Review*, 2013, 54: 14-31.
- [35] Zhen L, Lee L H, Chew E P. A decision model for berth allocation under uncertainty[J]. *European Journal of Operational Research*, 2011, 212(1): 54-68.
- [36] Legato P, Mazza R M. Berth planning and resources optimisation at a container terminal via discrete event simulation[J]. *European Journal of Operational Research*, 2001, 133(3): 537-547.
- [37] Moorthy R, Teo C P. Berth management in container terminal: the template design problem[J]. *OR Spectrum*, 2006, 28(4): 495-518.
- [38] Kizilay D, Eliyi D T, Van Hentenryck P. Constraint and mathematical programming models for integrated port container terminal operations[R]. Ithaca: arXiv, 2017.

附录 A 主要计算程序

该文件用于读取 CSV 数据、建立 Gurobi 工艺选择与设备增配 MILP，并输出 process_selection_model.lp、process_selection_solution.sol、gurobi.log、资源负荷表和备增配表。

```
from __future__ import annotations

import json
import shutil
import tempfile
from pathlib import Path

import pandas as pd

PACKAGE_DIR = Path(__file__).resolve().parent
DATA_DIR = PACKAGE_DIR / "data"
OUTPUT_DIR = PACKAGE_DIR / "outputs"
GUROBI_TMP_DIR = Path(tempfile.gettempdir()) / "port_process_selection_gurobi"

REQUIRED_CANDIDATE_COLUMNS = {
    "edge_id",
    "candidate_id",
    "flow_id",
    "cargo",
    "operation",
    "option_id",
    "equipment",
    "efficiency_tph",
    "work_hours",
    "labor_hours",
    "machine_hours_total",
    "machine_hours_json",
    "driver_hours_json",
    "ship_hours",
    "is_default",
}

REQUIRED_RESOURCE_COLUMNS = {
    "resource_id",
    "resource_type",
    "resource_name",
}
```

```

    "capacity",
    "unit",
    "fixed_demand",
}

REQUIRED_MACHINERY_COLUMNS = {
    "machine",
    "existing_units",
    "available_hours_total",
    "driver_hours_total",
}

def _read_json_dict(text: object) -> dict[str, float]:
    if pd.isna(text) or text == "":
        return {}
    return {str(k): float(v) for k, v in json.loads(str(text)).items()}

def _require_columns(df: pd.DataFrame, required: set[str], name: str) -> None:
    missing = sorted(required - set(df.columns))
    if missing:
        raise ValueError(f"{name} is missing required columns: {' '.join(missing)}")

def validate_candidate_options(candidates: pd.DataFrame) -> None:
    _require_columns(candidates, REQUIRED_CANDIDATE_COLUMNS, "candidate_options.csv")
    if candidates.empty:
        raise ValueError("candidate_options.csv is empty.")
    if candidates["candidate_id"].duplicated().any():
        duplicated = candidates.loc[candidates["candidate_id"].duplicated(),
        "candidate_id"].head(5).tolist()
        raise ValueError(f"candidate_id must be unique. Duplicates: {duplicated}")
    if candidates["edge_id"].isna().any() or candidates["candidate_id"].isna().any():
        raise ValueError("edge_id and candidate_id cannot contain null values.")
    for col in ["efficiency_tph", "work_hours", "labor_hours", "machine_hours_total",
    "ship_hours"]:
        if (pd.to_numeric(candidates[col], errors="coerce") < 0).any():
            raise ValueError(f"{col} cannot contain negative values.")
    for col in ["machine_hours_json", "driver_hours_json"]:
        for value in candidates[col].dropna():
            parsed = _read_json_dict(value)
            if any(v < 0 for v in parsed.values()):
                raise ValueError(f"{col} cannot contain negative resource hours.")

```

```

def validate_resource_capacities(resources: pd.DataFrame) -> None:
    _require_columns(resources, REQUIRED_RESOURCE_COLUMNS, "resource_capacities.csv")
    if resources.empty:
        raise ValueError("resource_capacities.csv is empty.")
    if resources["resource_id"].duplicated().any():
        duplicated = resources.loc[resources["resource_id"].duplicated(),
"resource_id"].head(5).tolist()
        raise ValueError(f"resource_id must be unique. Duplicates: {duplicated}")
    if (pd.to_numeric(resources["capacity"], errors="coerce") <= 0).any():
        raise ValueError("capacity must be positive for every resource.")
    fixed_yard = resources["resource_type"].eq("fixed_yard")
    if fixed_yard.any() and resources.loc[fixed_yard, "fixed_demand"].isna().any():
        raise ValueError("fixed_yard resources must provide fixed_demand.")

def validate_machinery(machinery: pd.DataFrame) -> None:
    _require_columns(machinery, REQUIRED_MACHINERY_COLUMNS, "machinery.csv")
    if machinery["machine"].duplicated().any():
        duplicated = machinery.loc[machinery["machine"].duplicated(), "machine"].head(5).tolist()
        raise ValueError(f"machine must be unique. Duplicates: {duplicated}")
    for col in ["existing_units", "available_hours_total", "driver_hours_total"]:
        if (pd.to_numeric(machinery[col], errors="coerce") < 0).any():
            raise ValueError(f"{col} cannot contain negative values.")

def validate_inputs(candidates: pd.DataFrame, resources: pd.DataFrame, machinery: pd.DataFrame) ->
None:
    validate_candidate_options(candidates)
    validate_resource_capacities(resources)
    validate_machinery(machinery)

def _resource_expression(gp, y, candidates: pd.DataFrame, resource: pd.Series):
    resource_type = resource["resource_type"]
    resource_name = resource["resource_name"]

    if resource_type == "machine":
        return gp.quicksum(
            _read_json_dict(row.machine_hours_json).get(resource_name, 0.0) * y[row.candidate_id]
            for row in candidates.itertuples(index=False)
        )
    if resource_type == "driver":

```

```

        return gp.quicksum(
            _read_json_dict(row.driver_hours_json).get(resource_name, 0.0) * y[row.candidate_id]
            for row in candidates.itertuples(index=False)
        )
    if resource_type == "berth":
        return gp.quicksum(float(row.ship_hours) * y[row.candidate_id] for row in
candidates.itertuples(index=False))
    if resource_type == "labor":
        return gp.quicksum(float(row.labor_hours) * y[row.candidate_id] for row in
candidates.itertuples(index=False))
    if resource_type == "fixed_yard":
        return gp.LinExpr(float(resource["fixed_demand"]))
    raise ValueError(f"Unknown resource type: {resource_type}")

def _unit_capacity_lookup(machinery: pd.DataFrame) -> dict[str, dict[str, float]]:
    rows: dict[str, dict[str, float]] = {}
    for row in machinery.itertuples(index=False):
        existing = max(float(row.existing_units), 1.0)
        rows[str(row.machine)] = {
            "existing_units": float(row.existing_units),
            "machine_unit_hours": float(row.available_hours_total) / existing,
            "driver_unit_hours": float(row.driver_hours_total) / existing,
        }
    return rows

def _write_validation_failure(exc: Exception) -> None:
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    (OUTPUT_DIR / "input_validation_error.txt").write_text(str(exc), encoding="utf-8")

def _prepare_gurobi_file_paths() -> dict[str, Path]:
    """Use an ASCII temp directory for files written directly by Gurobi.

    Some Windows/Gurobi combinations fail to write log/model/solution files
    when the absolute path contains Chinese characters. Python can copy those
    files back to the project output directory after Gurobi finishes.
    """
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    GUROBI_TMP_DIR.mkdir(parents=True, exist_ok=True)
    paths = {
        "tmp_log": GUROBI_TMP_DIR / "gurobi.log",
        "tmp_lp": GUROBI_TMP_DIR / "process_selection_model.lp",
    }

```

```

    "tmp_sol": GUROBI_TMP_DIR / "process_selection_solution.sol",
    "out_log": OUTPUT_DIR / "gurobi.log",
    "out_lp": OUTPUT_DIR / "process_selection_model.lp",
    "out_sol": OUTPUT_DIR / "process_selection_solution.sol",
}
for path in paths.values():
    if path.exists():
        path.unlink()
return paths

def _copy_gurobi_file(src: Path, dst: Path) -> None:
    if src.exists():
        shutil.copy2(src, dst)

def _safe_model_attr(model, attr_name: str, default: object = "NA") -> object:
    try:
        return getattr(model, attr_name)
    except Exception:
        return default

def solve() -> None:
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    for stale_name in ["input_validation_error.txt", "gurobi_environment_error.txt"]:
        stale_path = OUTPUT_DIR / stale_name
        if stale_path.exists():
            stale_path.unlink()
    candidates = pd.read_csv(DATA_DIR / "candidate_options.csv", encoding="utf-8-sig")
    resources = pd.read_csv(DATA_DIR / "resource_capacities.csv", encoding="utf-8-sig")
    machinery = pd.read_csv(DATA_DIR / "machinery.csv", encoding="utf-8-sig")

    try:
        validate_inputs(candidates, resources, machinery)
    except Exception as exc:
        _write_validation_failure(exc)
        raise

    try:
        import gurobipy as gp
        from gurobipy import GRB
    except Exception as exc: # pragma: no cover
        message = (

```

```

        "gurobipy import failed. The current machine usually needs a valid "
        "Gurobi installation, license, and GUROBI_HOME pointing to the win64 "
        f"installation folder. Original error: {exc}"
    )
    (OUTPUT_DIR / "gurobi_environment_error.txt").write_text(message, encoding="utf-8")
    raise RuntimeError(message) from exc

gurobi_files = _prepare_gurobi_file_paths()
unit_capacity = _unit_capacity_lookup(machinery)
model = gp.Model("port_process_selection_with_equipment_augmentation")
model.Params.OutputFlag = 1
model.Params.LogFile = str(gurobi_files["tmp_log"])
model.Params.TimeLimit = 120

y = {
    row.candidate_id: model.addVar(vtype=GRB.BINARY, name=f"y_{row.candidate_id}")
    for row in candidates.itertuples(index=False)
}
add_units = {
    row.machine: model.addVar(lb=0.0, vtype=GRB.INTEGER, name=f"z_add_machine_{idx:03d}")
    for idx, row in enumerate(machinery.itertuples(index=False), start=1)
}
U = model.addVar(lb=0.0, vtype=GRB.CONTINUOUS, name="U_original_capacity_max_utilization")

for edge_pos, (_edge_id, group) in enumerate(candidates.groupby("edge_id"), start=1):
    model.addConstr(
        gp.quicksum(y[cid] for cid in group["candidate_id"]) == 1,
        name=f"choose_edge_{edge_pos:03d}",
    )

overload = {}
resource_expr = {}
for res_pos, resource in enumerate(resources.itertuples(index=False), start=1):
    rid = resource.resource_id
    capacity = max(float(resource.capacity), 1e-6)
    resource_row = pd.Series(resource._asdict())
    expr = _resource_expression(gp, y, candidates, resource_row)
    resource_expr[rid] = expr

# U is measured against original capacity, so U>1 identifies the bottleneck level.
model.addConstr(expr <= capacity * U, name=f"util_res_{res_pos:03d}")
overload[rid] = model.addVar(lb=0.0, vtype=GRB.CONTINUOUS, name=f"over_res_{res_pos:03d}")
model.addConstr(expr - capacity <= overload[rid], name=f"overdef_res_{res_pos:03d}")

```

```

# Equipment augmentation creates an executable machine/driver plan.
if resource.resource_type == "machine" and resource.resource_name in add_units:
    machine = resource.resource_name
    model.addConstr(
        expr <= capacity + unit_capacity[machine]["machine_unit_hours"] *
add_units[machine],
        name=f"machine_aug_feas_{res_pos:03d}",
    )
elif resource.resource_type == "driver" and resource.resource_name in add_units:
    machine = resource.resource_name
    model.addConstr(
        expr <= capacity + unit_capacity[machine]["driver_unit_hours"] *
add_units[machine],
        name=f"driver_aug_feas_{res_pos:03d}",
    )

total_machine_hours = gp.quicksum(float(row.machine_hours_total) * y[row.candidate_id] for row
in candidates.itertuples(index=False))
total_labor_hours = gp.quicksum(float(row.labor_hours) * y[row.candidate_id] for row in
candidates.itertuples(index=False))
total_added_units = gp.quicksum(add_units.values())
overload_ratio = gp.quicksum(
    overload[row.resource_id] / max(float(row.capacity), 1e-6)
    for row in resources.itertuples(index=False)
)

model.setObjectiveN(U, index=0, priority=4, weight=1.0,
name="minimize_original_max_resource_utilization")
model.setObjectiveN(overload_ratio, index=1, priority=3, weight=1.0,
name="minimize_original_overload_ratio")
model.setObjectiveN(total_added_units, index=2, priority=2, weight=1.0,
name="minimize_added_equipment_units")
model.setObjectiveN(
    total_machine_hours / 1_000_000 + total_labor_hours / 1_000_000,
    index=3,
    priority=1,
    weight=1.0,
    name="minimize_hours",
)

model.write(str(gurobi_files["tmp_lp"]))
_copy_gurobi_file(gurobi_files["tmp_lp"], gurobi_files["out_lp"])
model.optimize()

```

```

summary_rows = [
    {"item": "status_code", "value": model.Status},
    {"item": "solution_count", "value": model.SolCount},
    {"item": "num_variables", "value": model.NumVars},
    {"item": "num_constraints", "value": model.NumConstrs},
    {"item": "model_scope", "value": "fixed routes/direct ratios; optimize y_ek and equipment
augmentation z_m"},
]

if model.SolCount <= 0:
    pd.DataFrame(summary_rows).to_csv(OUTPUT_DIR / "gurobi_model_summary.csv", index=False,
encoding="utf-8-sig")
    raise RuntimeError("Gurobi did not return a feasible solution. Check outputs/gurobi.log.")

model.write(str(gurobi_files["tmp_sol"]))
_copy_gurobi_file(gurobi_files["tmp_sol"], gurobi_files["out_sol"])
_copy_gurobi_file(gurobi_files["tmp_log"], gurobi_files["out_log"])
summary_rows.extend(
    [
        {"item": "objective_value", "value": _safe_model_attr(model, "ObjVal")},
        {"item": "max_original_resource_utilization_U", "value": U.X},
        {"item": "total_added_equipment_units", "value": sum(v.X for v in add_units.values())},
        {"item": "mip_gap", "value": _safe_model_attr(model, "MIPGap", "NA")},
        {"item": "mip_gap_note", "value": "MIPGap may be unavailable for this Gurobi multi-
objective solve; use status_code and solution_count for primary feasibility check."},
    ]
)

selected_rows = []
for edge_id, group in candidates.groupby("edge_id"):
    selected = max(group.itertuples(index=False), key=lambda row: y[row.candidate_id].X)
    selected_rows.append(
        {
            "edge_id": edge_id,
            "candidate_id": selected.candidate_id,
            "flow_id": selected.flow_id,
            "cargo": selected.cargo,
            "operation": selected.operation,
            "option_id": selected.option_id,
            "equipment": selected.equipment,
            "efficiency_tph": selected.efficiency_tph,
            "work_hours": selected.work_hours,
            "labor_hours": selected.labor_hours,
            "machine_hours_total": selected.machine_hours_total,
        }
    )

```

```

        "is_default": selected.is_default,
    }
)

resource_rows = []
resource_lookup = resources.set_index("resource_id")
for rid, expr in resource_expr.items():
    row = resource_lookup.loc[rid]
    demand = float(expr.getValue())
    capacity = float(row["capacity"])
    augmented_capacity = capacity
    if row["resource_type"] == "machine" and row["resource_name"] in add_units:
        machine = row["resource_name"]
        augmented_capacity += unit_capacity[machine]["machine_unit_hours"] *
float(add_units[machine].X)
    elif row["resource_type"] == "driver" and row["resource_name"] in add_units:
        machine = row["resource_name"]
        augmented_capacity += unit_capacity[machine]["driver_unit_hours"] *
float(add_units[machine].X)
    resource_rows.append(
        {
            "resource_id": rid,
            "resource_type": row["resource_type"],
            "resource_name": row["resource_name"],
            "demand": demand,
            "original_capacity": capacity,
            "augmented_capacity": augmented_capacity,
            "unit": row["unit"],
            "original_utilization": demand / capacity if capacity else 0.0,
            "augmented_utilization": demand / augmented_capacity if augmented_capacity else
0.0,

            "original_overload": max(0.0, demand - capacity),
            "remaining_overload_after_augmentation": max(0.0, demand - augmented_capacity),
        }
    )

equipment_rows = []
machinery_lookup = machinery.set_index("machine")
for machine, var in add_units.items():
    existing = float(machinery_lookup.loc[machine, "existing_units"])
    machine_resource_id = f"machine_{machine}"
    driver_resource_id = f"driver_{machine}"
    machine_demand = float(resource_expr[machine_resource_id].getValue()) if
machine_resource_id in resource_expr else 0.0

```

```

        driver_demand = float(resource_expr[driver_resource_id].getValue()) if driver_resource_id
in resource_expr else 0.0
    equipment_rows.append(
        {
            "machine": machine,
            "existing_units": existing,
            "added_units_z_m": round(float(var.X)),
            "units_after_augmentation": existing + round(float(var.X)),
            "machine_unit_available_hours": unit_capacity[machine]["machine_unit_hours"],
            "driver_unit_available_hours": unit_capacity[machine]["driver_unit_hours"],
            "machine_hour_demand": machine_demand,
            "driver_hour_demand": driver_demand,
        }
    )

    pd.DataFrame(selected_rows).to_csv(OUTPUT_DIR / "gurobi_selected_options.csv", index=False,
encoding="utf-8-sig")
    pd.DataFrame(resource_rows).sort_values("original_utilization", ascending=False).to_csv(
        OUTPUT_DIR / "gurobi_resource_loads.csv",
        index=False,
        encoding="utf-8-sig",
    )
    pd.DataFrame(equipment_rows).sort_values("added_units_z_m", ascending=False).to_csv(
        OUTPUT_DIR / "gurobi_equipment_augmentation.csv",
        index=False,
        encoding="utf-8-sig",
    )
    pd.DataFrame(summary_rows).to_csv(OUTPUT_DIR / "gurobi_model_summary.csv", index=False,
encoding="utf-8-sig")
    print(f"Gurobi process-selection outputs written to: {OUTPUT_DIR}")

if __name__ == "__main__":
    solve()

```

该文件用于运行确定性贪心负荷均衡算法，作为 MILP 求解结果的对照。

```

from __future__ import annotations

import json
from pathlib import Path

import pandas as pd

```

```

PACKAGE_DIR = Path(__file__).resolve().parent
DATA_DIR = PACKAGE_DIR / "data"
OUTPUT_DIR = PACKAGE_DIR / "outputs"

```

```

REQUIRED_CANDIDATE_COLUMNS = {
    "edge_id",
    "candidate_id",
    "flow_id",
    "cargo",
    "operation",
    "option_id",
    "equipment",
    "efficiency_tph",
    "work_hours",
    "labor_hours",
    "machine_hours_total",
    "machine_hours_json",
    "driver_hours_json",
    "ship_hours",
    "is_default",
}

```

```

REQUIRED_RESOURCE_COLUMNS = {
    "resource_id",
    "resource_type",
    "resource_name",
    "capacity",
    "unit",
    "fixed_demand",
}

```

```

def _read_json_dict(text: object) -> dict[str, float]:
    if pd.isna(text) or text == "":
        return {}
    return {str(k): float(v) for k, v in json.loads(str(text)).items()}

```

```

def _require_columns(df: pd.DataFrame, required: set[str], name: str) -> None:
    missing = sorted(required - set(df.columns))
    if missing:
        raise ValueError(f"{name} is missing required columns: {' '.join(missing)}")

```

```

def validate_inputs(candidates: pd.DataFrame, resources: pd.DataFrame) -> None:
    _require_columns(candidates, REQUIRED_CANDIDATE_COLUMNS, "candidate_options.csv")
    _require_columns(resources, REQUIRED_RESOURCE_COLUMNS, "resource_capacities.csv")
    if candidates.empty:
        raise ValueError("candidate_options.csv is empty.")
    if resources.empty:
        raise ValueError("resource_capacities.csv is empty.")
    if candidates["candidate_id"].duplicated().any():
        raise ValueError("candidate_id must be unique.")
    if resources["resource_id"].duplicated().any():
        raise ValueError("resource_id must be unique.")
    if (pd.to_numeric(resources["capacity"], errors="coerce") <= 0).any():
        raise ValueError("capacity must be positive for every resource.")

def _empty_loads(resources: pd.DataFrame) -> dict[str, float]:
    loads = {row.resource_id: 0.0 for row in resources.itertuples(index=False)}
    for row in resources.itertuples(index=False):
        if row.resource_type == "fixed_yard":
            loads[row.resource_id] = float(row.fixed_demand)
    return loads

def _add_candidate_load(loads: dict[str, float], resources: pd.DataFrame, candidate: pd.Series,
sign: float = 1.0) -> None:
    machine_hours = _read_json_dict(candidate["machine_hours_json"])
    driver_hours = _read_json_dict(candidate["driver_hours_json"])
    for row in resources.itertuples(index=False):
        if row.resource_type == "machine":
            loads[row.resource_id] += sign * machine_hours.get(row.resource_name, 0.0)
        elif row.resource_type == "driver":
            loads[row.resource_id] += sign * driver_hours.get(row.resource_name, 0.0)
        elif row.resource_type == "berth":
            loads[row.resource_id] += sign * float(candidate["ship_hours"])
        elif row.resource_type == "labor":
            loads[row.resource_id] += sign * float(candidate["labor_hours"])

def _score(loads: dict[str, float], resources: pd.DataFrame, selected: pd.DataFrame) ->
tuple[float, float, float]:
    capacity = resources.set_index("resource_id")["capacity"].to_dict()
    utilizations = [loads[rid] / max(float(capacity[rid]), 1e-6) for rid in loads]
    overload_ratio = sum(max(0.0, loads[rid] - float(capacity[rid])) /

```

```

max(float(capacity[rid]), 1e-6) for rid in loads)
    total_hours = float(selected["machine_hours_total"].sum() + selected["labor_hours"].sum())
    return (max(utilizations), overload_ratio, total_hours)

def solve() -> None:
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    candidates = pd.read_csv(DATA_DIR / "candidate_options.csv", encoding="utf-8-sig")
    resources = pd.read_csv(DATA_DIR / "resource_capacities.csv", encoding="utf-8-sig")
    validate_inputs(candidates, resources)

    selected_rows = []
    for _, group in candidates.groupby("edge_id", sort=False):
        default_rows = group[group["is_default"].astype(str).str.lower().isin(["true", "1"])]
        if default_rows.empty:
            chosen = group.sort_values(["default_rank", "machine_hours_total",
"labor_hours"]).iloc[0]
        else:
            chosen = default_rows.iloc[0]
        selected_rows.append(chosen)
    selected = pd.DataFrame(selected_rows).reset_index(drop=True)

    loads = _empty_loads(resources)
    for _, row in selected.iterrows():
        _add_candidate_load(loads, resources, row, sign=1.0)
    best_score = _score(loads, resources, selected)

    improved = True
    iterations = 0
    while improved and iterations < 100:
        improved = False
        iterations += 1
        for edge_id, group in candidates.groupby("edge_id", sort=False):
            current_idx = selected.index[selected["edge_id"] == edge_id][0]
            current = selected.loc[current_idx]
            for _, candidate in group.iterrows():
                if candidate["candidate_id"] == current["candidate_id"]:
                    continue
                trial_loads = dict(loads)
                _add_candidate_load(trial_loads, resources, current, sign=-1.0)
                _add_candidate_load(trial_loads, resources, candidate, sign=1.0)
                trial_selected = selected.copy()
                trial_selected.loc[current_idx] = candidate
                trial_score = _score(trial_loads, resources, trial_selected)

```

```

        if trial_score < best_score:
            selected = trial_selected
            loads = trial_loads
            best_score = trial_score
            improved = True
            break
    if improved:
        break

capacity = resources.set_index("resource_id")["capacity"].to_dict()
resource_meta = resources.set_index("resource_id")
resource_rows = []
for rid, demand in loads.items():
    meta = resource_meta.loc[rid]
    cap = float(capacity[rid])
    resource_rows.append(
        {
            "resource_id": rid,
            "resource_type": meta["resource_type"],
            "resource_name": meta["resource_name"],
            "demand": demand,
            "capacity": cap,
            "unit": meta["unit"],
            "utilization": demand / cap if cap else 0.0,
            "overload": max(0.0, demand - cap),
        }
    )

selected.to_csv(OUTPUT_DIR / "heuristic_selected_options.csv", index=False, encoding="utf-8-sig")
pd.DataFrame(resource_rows).sort_values("utilization", ascending=False).to_csv(
    OUTPUT_DIR / "heuristic_resource_loads.csv",
    index=False,
    encoding="utf-8-sig",
)
pd.DataFrame(
    [
        {"item": "iterations", "value": iterations},
        {"item": "max_resource_utilization", "value": best_score[0]},
        {"item": "overload_ratio", "value": best_score[1]},
        {"item": "total_machine_labor_hours", "value": best_score[2]},
    ]
).to_csv(OUTPUT_DIR / "heuristic_summary.csv", index=False, encoding="utf-8-sig")
print(f"Heuristic outputs written to: {OUTPUT_DIR}")

```

```
if __name__ == "__main__":
    solve()
```

附录 B 货流拆分明细表

表 B-1 货流拆分明细表

flow_id	原始流向	货类	拆分比例	拆分后货量_万 吨	路径	备注
F001	国内出口	杂货	1.00	41.77	车—场—船	集装箱杂货
F002	国内出口	生铁	0.60	64.44	车—场—船	电磁吸盘卸车 60%，后续成组装船
F003	国内出口	生铁	0.40	42.96	车—场—船	非电磁吸盘部分按成组货处理
F004	国内出口	水泥	1.00	17.90	车—库—船	卸车后全部货板
F005	国内出口	纯碱	1.00	22.67	车—场—船	成组
F006	国内进口	棉花	1.00	2.86	船—车	直接换装
F007	国内进口	矿石	0.50	21.48	船—车	直接换装
F008	国内进口	矿石	0.50	21.48	船—场—车	经堆场疏运
F009	国内进口	杂货	0.25	3.10	船—车	直接换装，不成组
F010	国内进口	杂货	0.75	9.31	船—库—车	入库部分全部货盘成组
F011	国外出口	海盐	0.50	5.73	车—船	直接换装
F012	国外出口	海盐	0.50	5.73	车—场—船	网络成组后装船
F013	国外出口	大米	1.00	21.48	车—库—船	卸车后网络成组
F014	国外出口	杂货	1.00	31.74	车—场—船	集装箱杂货
F015	国外进口	橡胶	1.00	5.97	船—船	直接过驳
F016	国外进口	钢材	0.30	42.96	船—车	直接换装
F017	国外进口	钢材	0.70	100.24	船—场—车	经堆场疏运
F018	国外进口	其它	0.90	34.37	船—库—车	任务书给定船—库—车，90%按整箱经库区疏运处理
F019	国外进口	其它	0.10	3.82	船—库—车	任务书备注 10%拆箱后汽车疏运，库—车边套用件杂货疏运工艺

附录 C 机械台时统计表

表 C-1 机械台时统计表

机类	现有台数	计划期所需台时	单台可用台时	需要台数	机械利用率	平均台时效率_t_h	是否短缺
门机	25	89,780	2227.29	40.31	161.24%	54.42	是
桥吊	3	6443.91	2227.29	2.89	96.44%	100.00	否
装卸桥	2	5584.73	2227.29	2.51	125.37%	200.00	是
场桥	4	5393.79	2004.56	2.69	67.27%	200.00	否
K 吊	20	19,494	1781.83	10.94	54.70%	72.60	否
柴吊	8	0.00	1781.83	0.00	0.00%	0.00	否
正面吊	2	0.00	1781.83	0.00	0.00%	0.00	否
跨运车	2	11,169	1559.10	7.16	358.20%	100.00	是
拖挂车	4	0.00	1781.83	0.00	0.00%	0.00	否
万能机	12	61,416	1781.83	34.47	287.23%	29.92	是
大叉车	19	0.00	1781.83	0.00	0.00%	0.00	否
小叉车	15	32,884	1781.83	18.46	123.04%	30.77	是
大黄斗	2	1431.98	1781.83	0.80	40.18%	150.00	否
拖头	10	44,102	1781.83	24.75	247.51%	32.90	是
推扒机	2	3068.53	1781.83	1.72	86.11%	14.00	否

附录 D 通过能力核算表

表 D-1 通过能力核算表

环节	需求量	可供能力	利用率	计算口径
泊位系统	5,000,000	5,125,950	97.54%	16 个泊位×营运天数×昼夜小时×船边平均作业线效率×K 利×(1-μ)
仓库	868,735	560,175	155.08%	A 库×a 库×K 利×T 库/t 库
前方堆场	2,575,179	1,470,000	175.18%	A 场前×a 场×K 利×T 场/t 场
后方堆场	735,084	2,450,000	30.00%	A 场后×a 场×K 利×T 场/t 场
装卸工人	996,674	7,140,000	13.96%	1000 人×T 场×n 班×8 小时×K 出